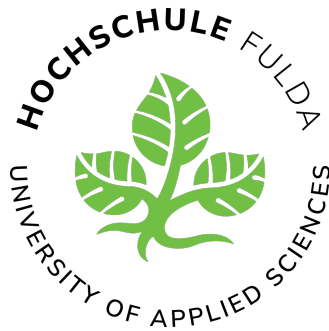




Praxisorientierte Fallstudie:

Nutzwertanalyse von End-to-End-Test-Frameworks (Selenium vs. Cypress) für die Automatisierung repräsentativer regressionskritischer Testszenarien zur Reduzierung des manuellen Testaufwands

Name: Marcel Licht

Studiengang: Bachelor Angewandte Informatik dual

Modul: Aktuelles Thema der Angewandten Informatik – Praxisorientierte Fallstudie

Matrikelnummer: 1540372

Lehrperson: Prof. Dr. Martine Herpers

Unternehmen: JUMO GmbH & Co. KG

Fachbetreuung: Daniel Kircher

Datum: 13. August 2026

Zusammenfassung

Diese Fallstudie untersucht die Automatisierung von End-to-End-Tests für die interne Webanwendung „Puma“ der JUMO GmbH & Co. KG, um den zeitintensiven, manuellen Testaufwand im Entwicklungsalltag zu reduzieren. Im Mittelpunkt der Untersuchung steht der strukturierte Vergleich der beiden Test-Frameworks Selenium und Cypress hinsichtlich ihrer Eignung in einer stark reglementierten Unternehmens-Infrastruktur. Als Methodik diente die prototypische Implementierung von drei repräsentativen, regressionskritischen Testszenarien (Login, Formularverarbeitung und Datensatzbearbeitung). Die daraus gewonnenen praktischen Erkenntnisse wurden anschließend mittels einer Nutzwertanalyse anhand von fünf praxisnahen Kriterien systematisch ausgewertet. Die Ergebnisse der Analyse zeigen, dass Cypress dem Framework Selenium aus rein entwicklungstechnischer Sicht überlegen ist. Cypress überzeugt insbesondere durch eine intuitivere Syntax und eine signifikant höhere Teststabilität durch integriertes Auto-Waiting. Einer sofortigen Einführung stehen jedoch Hürden im Weg, da strenge Proxy-Vorgaben und das etablierte Java-Ökosystem des Unternehmens die benötigte Node.js-Umgebung blockieren. Die Arbeit kommt zu der Handlungsempfehlung, kurzfristig Selenium für die Automatisierung zu nutzen, da es sich als Java-Bibliothek reibungslos in den bestehenden Build-Prozess integrieren lässt. Mittelfristig wird dem Unternehmen jedoch geraten, die IT-Infrastruktur für Cypress zu öffnen, um von dessen technischen Vorteilen zu profitieren.

Inhaltsverzeichnis

Abbildungsverzeichnis	4
Tabellenverzeichnis	5
Quellcodeverzeichnis	6
1 Einleitung	7
1.1 Problemstellung und Motivation	7
1.2 Zielsetzung der Arbeit	7
1.3 Forschungsfrage und Aufbau der Arbeit	8
2 Theoretische Grundlagen	9
2.1 Stand der Forschung	9
2.2 Softwaretest und End-to-End-Testing	9
2.3 Betrachtete Test-Frameworks: Selenium und Cypress	9
2.4 Nutzwertanalyse als Bewertungsmethode	10
3 Methodisches Vorgehen	11
3.1 Forschungsdesign der Fallstudie	11
3.2 Auswahl repräsentativer regressionskritischer Testszenarien	11
3.3 Ableitung und Gewichtung der Bewertungskriterien	12
3.4 Vorgehen der prototypischen Umsetzung	12
3.5 Datenerhebung und Auswertung	13
4 Implementierung und Durchführung der Prototypen	14
4.1 Testumgebung und Rahmenbedingungen	14
4.2 Beschreibung der Testszenarien	15
4.2.1 Szenario 1: Login	15
4.2.2 Szenario 2: Neues Projekt bzw. neue Aktivität anlegen	16
4.2.3 Szenario 3: Aktivität bearbeiten	16
4.3 Setup und technische Integration	16
4.3.1 Selenium	17
4.3.2 Cypress	17
4.4 Syntax- und Architekturvergleich	17
4.4.1 Architektur	17
4.4.2 Ausdrucksstärke der Syntax	17
4.4.3 Wartbarkeit	18
4.4.4 Synchronisation	18

4.4.5	Benutzeroberfläche und Test-Logging	18
4.5	Code-Gegenüberstellung: Projektanlage	19
5	Evaluation und Nutzwertanalyse	22
5.1	Durchführung und Testergebnisse	22
5.2	Auswertung des Kriterienmodells	22
5.2.1	Integration in die Infrastruktur (Gewichtung: 25 %)	23
5.2.2	Lernkurve und Syntax (Gewichtung: 20 %)	23
5.2.3	Test-Ausführung (Gewichtung: 20 %)	24
5.2.4	Stabilität (Gewichtung: 20 %)	24
5.2.5	Dokumentation und Community (Gewichtung: 15 %)	24
5.3	Nutzwertanalyse und Gesamtauswertung	25
6	Fazit und Ausblick	27
6.1	Beantwortung der Forschungsfrage und Handlungsempfehlung	27
6.2	Kritische Reflexion und Methodengrenzen	28
6.3	Ausblick	28
	Literaturverzeichnis	29
	Glossar	29
	Abkürzungsverzeichnis	30
	Eigenständigkeitserklärung	30
	Anhänge	31
A	Erklärung zur Nutzung von KI-Werkzeugen	31
B	Bewertungsmatrix (Excel)	31
C	Testprotokolle (Testlogs)	33
C.1	Testprotokolle: Selenium	33
C.2	Testprotokolle: Cypress	45

Abbildungsverzeichnis

1	Fachliche und technische Struktur der Webanwendung Puma	15
2	Grafische Gegenüberstellung der Teil- und Gesamtnutzwerte	26
3	Bewertungsmatrix der Nutzwertanalyse (Selenium vs. Cypress) – Teil 1 .	31
4	Bewertungsmatrix der Nutzwertanalyse (Selenium vs. Cypress) – Teil 2 .	32

Tabellenverzeichnis

1	Ergebnis der Nutzwertanalyse gemäß Bewertungsmatrix	25
---	---	----

Quellcodeverzeichnis

1	Projektanlage mit Selenium	20
2	Projektanlage in Cypress	20
3	Selenium – Login (Erfolgreich)	33
4	Selenium – Neues Projekt (Exemplarische Fehlerausgabe)	33
5	Selenium – Neues Projekt (Erfolgreich)	33
6	Selenium – Neue Activity (Erfolgreich)	34
7	Selenium – Activity bearbeiten (Exemplarische Fehlerausgabe)	37
8	Selenium – Activity bearbeiten (Erfolgreich)	38
9	Selenium – Kontrolllauf Activity bearbeiten (Erfolgreich)	40
10	Cypress – Login (Erfolgreich)	45
11	Cypress – Neues Projekt (Erfolgreich)	45
12	Cypress – Neue Activity (Erfolgreich)	47
13	Cypress – Activity bearbeiten & kontrollieren (Erfolgreich)	49

1 Einleitung

1.1 Problemstellung und Motivation

Das Testen von Software ist ein zentraler Bestandteil der Softwareentwicklung. Es stellt sicher, dass Anwendungen den funktionalen Anforderungen entsprechen und zuverlässig arbeiten. Durch Tests lassen sich Fehler frühzeitig erkennen und Probleme vor dem produktiven Einsatz reduzieren.

Softwaretests umfassen verschiedene Testarten und Vorgehensweisen. Während Tests früher häufig manuell durch direkte Interaktion mit der Anwendung durchgeführt wurden, hat sich aufgrund steigender Anforderungen an Qualität und Entwicklungsgeschwindigkeit der Einsatz automatisierter Testverfahren etabliert [1].

Im Kontext der JUMO GmbH & Co. KG wird eine interne Webanwendung (Puma) fortlaufend durch Fehlerbehebungen und funktionale Erweiterungen weiterentwickelt. Trotz der bekannten Vorteile der Automatisierung erfolgen Prüfungen zentraler Nutzungsszenarien derzeit noch manuell, häufig in Form explorativer Tests.

Dieser manuelle Testaufwand ist zeitintensiv, bindet personelle Ressourcen und schränkt die Reproduzierbarkeit der Testergebnisse ein, da die Ausführung stark von der testenden Person abhängt. Daher besteht der betriebliche Bedarf, Ansätze zur Automatisierung regressionskritischer End-to-End-Tests systematisch zu untersuchen. Dies betrifft speziell solche Testszenarien, die nach Änderungen besonders anfällig für Fehler oder unerwünschte Seiteneffekte sind.

Im Fokus dieser Fallstudie stehen die Test-Frameworks Selenium [2] und Cypress [3]. Beide Werkzeuge automatisieren browserbasierte Tests, unterscheiden sich jedoch in ihrer technischen Architektur, der Integration in bestehende Entwicklungsumgebungen, der Wartbarkeit der Testimplementierung sowie den infrastrukturellen Voraussetzungen.

Die Motivation dieser Arbeit ergibt sich aus dem konkreten Bedarf, die Qualitätssicherung bei JUMO im Kontext häufiger Änderungen effizienter zu gestalten. Ziel ist es nicht, ein allgemein überlegenes Framework zu bestimmen, sondern das für diesen spezifischen Unternehmenskontext am besten geeignete Werkzeug durch eine Nutzwertanalyse zu identifizieren.

1.2 Zielsetzung der Arbeit

Ziel der vorliegenden Fallstudie ist es, auf Basis eines strukturierten Vergleichs eine fundierte Entscheidungshilfe für die Auswahl eines End-to-End-Test-Frameworks bei der JUMO GmbH & Co. KG zu erarbeiten.

Dabei ist es nicht das Ziel, ein allgemein überlegenes Framework zu bestimmen, sondern das Werkzeug zu identifizieren, welches für den konkreten Unternehmenskontext und dessen technische Restriktionen am besten geeignet ist. Hierzu werden relevante Anforderungen aus dem Betrieb abgeleitet, ausgewählte Nutzungsszenarien der „Puma“-

Webanwendung prototypisch in beiden Frameworks umgesetzt und die Ergebnisse anhand einer Nutzwertanalyse systematisch bewertet.

1.3 Forschungsfrage und Aufbau der Arbeit

Aus der formulierten Zielsetzung leitet sich die zentrale Forschungsfrage dieser Fallstudie ab:

Welches der beiden Frameworks (Selenium oder Cypress) ist für die Automatisierung ausgewählter repräsentativer regressionskritischer End-to-End-Testszenarien im gegebenen Unternehmenskontext besser geeignet, wenn Integrationsaufwand, Wartbarkeit, Stabilität, Ausführungseffizienz und Unterstützungsangebote systematisch berücksichtigt werden?

Zur präzisen Beantwortung der Hauptfrage werden folgende Teilfragen untersucht:

- Welche Anforderungen ergeben sich aus dem betrieblichen Umfeld an ein End-to-End-Test-Framework?
- Welche Nutzungsszenarien der Webanwendung eignen sich als repräsentative Testfälle für einen Framework-Vergleich?
- Wie unterscheiden sich Selenium und Cypress hinsichtlich des technischen und organisatorischen Integrationsaufwands?
- Welche Unterschiede zeigen sich in Bezug auf Verständlichkeit, Wartbarkeit und Änderbarkeit des Testcodes?
- Wie unterscheiden sich die Frameworks bezüglich Stabilität, Laufzeit und Diagnosemöglichkeiten im Fehlerfall?

Aufbau der Arbeit:

Im Anschluss an diese Einleitung werden in Kapitel 2 die theoretischen Grundlagen des End-to-End-Testings sowie der untersuchten Frameworks Selenium und Cypress erläutert. Kapitel 3 widmet sich der Methodik und beschreibt das Vorgehen der Nutzwertanalyse inklusive der Entwicklung des Kriterienmodells. In Kapitel 4 erfolgt die Beschreibung der prototypischen Umsetzung und der Datenerhebung anhand der ausgewählten Testszenarien. Kapitel 5 präsentiert die Auswertung der Ergebnisse mittels der Nutzwertanalyse. Die Arbeit schließt in Kapitel 6 mit einer Diskussion der Erkenntnisse und einem zusammenfassenden Fazit.

2 Theoretische Grundlagen

Dieses Kapitel legt das theoretische Fundament für die Fallstudie. Es beleuchtet zunächst den aktuellen Stand der Forschung, definiert die Rolle von End-to-End-Tests in der Qualitätssicherung, stellt anschließend die beiden zu vergleichenden Frameworks vor und erläutert abschließend die Methodik der Nutzwertanalyse.

2.1 Stand der Forschung

Die Evaluierung und der Vergleich von Testautomatisierungswerkzeugen sind Gegenstand zahlreicher Publikationen der angewandten Informatik. Während grundlegende Werke wie Liggesmeyer [4] oder Spillner und Linz [1] die theoretischen Konzepte des Softwaretests und der Testautomatisierung umfassend behandeln, fokussieren sich neuere Publikationen häufig auf den Architekturvergleich spezifischer Frameworks. Der Kontrast zwischen traditionellen WebDriver-basierten Ansätzen wie Selenium und modernen, browserinternen Werkzeugen wie Cypress wird in der Fachliteratur oft anhand von Metriken wie Ausführungsgeschwindigkeit, Stabilität (Flakiness) und API-Design diskutiert.

Eine Lücke besteht jedoch oft in der detaillierten Betrachtung dieser Tools innerhalb stark reglementierter Unternehmensnetzwerke und spezifischer Legacy-Infrastrukturen (wie z. B. ältere JDK-Versionen oder strikte Proxy-Vorgaben). Die vorliegende Fallstudie knüpft an den bestehenden Forschungsstand an, erweitert ihn jedoch um eine systematische Bewertung, die exakt diese praxisnahen infrastrukturellen und organisatorischen Rahmenbedingungen der JUMO GmbH & Co. KG in den Mittelpunkt stellt.

2.2 Softwaretest und End-to-End-Testing

Die Testautomatisierung ist ein fest etablierter Bestandteil moderner Softwarequalitätssicherung [4]. Während Unit- und Integrationstests isolierte Code-Abschnitte oder Schnittstellen auf technischer Ebene prüfen, testen End-to-End-Tests (E2E-Tests) das gesamte System über alle Schichten hinweg – von der Benutzeroberfläche bis zur Datenbank. Sie dienen insbesondere dazu, regressionskritische Nutzungsszenarien aus Sicht der Endanwender in einer realitätsnahen Umgebung zu überprüfen und dadurch zentrale Geschäftsprozesse abzusichern [1]. Da E2E-Tests in der Regel komplexer in der Einrichtung und laufzeitintensiver sind, ist die Wahl eines robusten, wartbaren und in die Systemlandschaft passenden Automatisierungs-Frameworks entscheidend für die langfristige Akzeptanz im Entwicklungsalltag.

2.3 Betrachtete Test-Frameworks: Selenium und Cypress

Für den praktischen Vergleich dieser Arbeit stehen die Werkzeuge Selenium und Cypress im Mittelpunkt.

Selenium ist ein langjährig etabliertes Standardwerkzeug der browserbasierten Testautomatisierung. Es steuert den Browser über das native WebDriver-Protokoll an und bietet breite Integrationsmöglichkeiten in unterschiedliche Programmiersprachen, wie

beispielsweise Java [2]. Diese hohe Flexibilität und Rückwärtskompatibilität erfordern jedoch im Gegenzug oftmals einen höheren manuellen Konfigurationsaufwand, insbesondere bei der Handhabung von asynchronen Ladezeiten und der Synchronisation des Testablaufs.

Cypress wird demgegenüber häufig als moderne, entwicklerzentrierte Alternative für Webanwendungen herangezogen. Im Gegensatz zu Selenium läuft Cypress direkt im Browser und führt Tests in derselben Event-Loop wie die Anwendung selbst aus [3]. Dies ermöglicht eine direkte Rückmeldung während der Testausführung, ein automatisches Warten auf DOM-Elemente (*Auto-Waiting*) sowie sehr gute Debugging-Möglichkeiten. Da Cypress jedoch primär auf JavaScript beziehungsweise TypeScript ausgelegt ist, bedarf es bei der Integration in bestehende Java-Ökosysteme oder stark reglementierte Firmennetzwerke einer gesonderten Betrachtung.

Der Mehrwert der vorliegenden Arbeit liegt demnach nicht in einer rein allgemeinen Toolbeschreibung, sondern in der kontextbezogenen Bewertung dieser Frameworks für konkrete Webanwendungen unter realen infrastrukturellen Restriktionen im Unternehmensumfeld.

2.4 Nutzwertanalyse als Bewertungsmethode

Zur strukturierten und objektiven Entscheidungsfindung wird die Nutzwertanalyse verwendet. Diese Methode erlaubt es, mehrere Handlungsalternativen anhand systematisch gewichteter qualitativer und quantitativer Kriterien nachvollziehbar zu vergleichen [5]. Das Verfahren gliedert sich in vier zentrale Schritte: die Definition der relevanten Bewertungskriterien, deren Gewichtung auf Basis der betrieblichen Anforderungen, die Ermittlung der Zielerfüllung (Punktevergabe) durch prototypische Messungen sowie die finale Berechnung des Gesamtnutzwertes. Damit wird ein bestehender praktischer Bedarf mit einer methodisch fundierten Bewertungslogik verbunden, um eine kontextbezogene Entscheidungshilfe abzuleiten.

3 Methodisches Vorgehen

Um die Eignung der Frameworks Selenium und Cypress für den Einsatz bei der JUMO GmbH & Co. KG systematisch und objektiv zu vergleichen, wurde ein praxisorientiertes Forschungsdesign gewählt. Dieses Kapitel beschreibt den methodischen Aufbau der Fallstudie, die Auswahl der Testszenarien, die Definition der Bewertungskriterien sowie den Ablauf der praktischen Erprobung.

3.1 Forschungsdesign der Fallstudie

Die Untersuchung basiert auf einer Kombination aus prototypischer Implementierung und einer strukturierten Entscheidungsfindung mittels Nutzwertanalyse (NWA) nach Kühnapfel [5]. Da ein rein theoretischer Vergleich der Frameworks den spezifischen infrastrukturellen Kontext des Unternehmens nicht ausreichend berücksichtigen würde, werden beide Werkzeuge in der realen Entwicklungsumgebung erprobt.

Die Ergebnisse dieser praktischen Umsetzung fließen in die Nutzwertanalyse ein. Diese Methode erlaubt es, qualitative und quantitative Beobachtungen in messbare Punktwerte zu übersetzen, diese mit unternehmensspezifischen Gewichtungen zu versehen und so zu einem nachvollziehbaren Gesamtergebnis zu gelangen.

3.2 Auswahl repräsentativer regressionskritischer Testszenarien

Da eine vollständige Automatisierung aller Anwendungsfälle der „Puma“-Webanwendung den Rahmen dieser Studie sprengen würde, wurden repräsentative und regressionskritische Nutzungsszenarien ausgewählt. Diese Szenarien bilden die Kernfunktionalitäten der Anwendung ab und sind nach funktionalen Erweiterungen besonders fehleranfällig.

Für die prototypische Umsetzung wurden folgende drei Szenarien definiert:

1. **Authentifizierung:** Ein Login-Vorgang zur Prüfung des Zugriffs auf geschützte Bereiche. Obwohl dieser Bereich bei funktionalen Erweiterungen der Anwendung selten direkt modifiziert wird, ist er aufgrund seiner hohen Kritikalität (Totalausfall bei Defekt) zwingend abzusichern. Zudem dient er im Framework-Vergleich als methodische Baseline zur Evaluation grundlegender Interaktions- und Wartemechanismen.
2. **Formularverarbeitung:** Das Ausfüllen, Validieren und Absenden eines typischen Eingabeformulars (Projekt-/Aktivitätsanlage). Dieses Szenario vergleicht, wie die Frameworks mit verschiedenen dynamischen UI-Elementen (z. B. Select2-Dropdowns) umgehen.
3. **Datensatzbearbeitung:** Ein mehrstufiger Prozess, bei dem ein Datensatz aufgerufen, modifiziert, gespeichert und verifiziert wird. Dies testet primär, wie zuverlässig die Werkzeuge asynchrone Ladezeiten und die zusammenhängende Navigation handhaben.

3.3 Ableitung und Gewichtung der Bewertungskriterien

Die Bewertungskriterien für die Nutzwertanalyse wurden direkt aus den fachlichen, technischen und organisatorischen Anforderungen des Ausbildungsbetriebs abgeleitet. Die Gewichtung (in Klammern) erfolgt vor dem Hintergrund der Unternehmensziele. Um eine hohe Transparenz und Nachvollziehbarkeit der Punktevergabe zu gewährleisten, wurde jedes Hauptkriterium in spezifische Unterkriterien aufgeschlüsselt:

- **Integration in die Infrastruktur (25 %):** Bewertung der initialen Einrichtung. Ein Test-Framework im betrieblichen Umfeld der JUMO GmbH & Co. KG kann nur effizient skaliert werden, wenn es die strengen Proxy-Vorgaben und das etablierte Java-Ökosystem respektiert. Ein zu hoher administrativer Overhead negiert den Automatisierungsgewinn, weshalb dieses Kriterium am höchsten gewichtet wird. *Zur feingranularen Bewertung unterteilt sich dieses Kriterium in die Aspekte Projektanlage & Tool-Start (40 %), Debugging & Ausführung (30 %) sowie CI/Reporting-Anschlussfähigkeit (30 %).*
- **Lernkurve und Syntax (20 %):** Beurteilung, wie intuitiv die Erstellung und Pflege von Tests ist, um eine schnelle Einarbeitung neuer Entwickler zu gewährleisten. *Die Bewertung setzt sich aus dem Aufwand für den ersten Test (40 %), der allgemeinen Usability (10 %), der Lesbarkeit & Wartbarkeit des Codes (25 %) sowie der Qualität der Fehlermeldungen und Diagnosemöglichkeiten (25 %) zusammen.*
- **Test-Ausführung (20 %):** Analyse der Ausführungsgeschwindigkeit im Entwicklungsalltag. Schnelle Feedback-Zyklen sind essenziell für die Akzeptanz bei den Entwicklern. *Hierbei werden die lokale Ausführungsgeschwindigkeit (60 %) und die theoretische Skalierbarkeit in einer CI-Pipeline (40 %) betrachtet.*
- **Stabilität (20 %):** Zuverlässigkeit der Tests zur Minimierung von *Flaky Tests*. Laufzeit und Stabilität bestimmen direkt den Erfolg der täglichen Qualitätssicherung. *Die Unterkriterien umfassen das systemseitige Warten & die Synchronisation dynamischer Inhalte (40 %), die Zuverlässigkeit über mehrere Testläufe hinweg (55 %) sowie die integrierten Debug-Hilfen im Fehlerfall (5 %).*
- **Dokumentation und Community (15 %):** Bewertung der offiziellen Hilfen und des Ökosystems. Dies flankiert den Entwicklungsprozess, ist den technischen Kernaspekten jedoch untergeordnet. *Dieses Kriterium gliedert sich in die Doku-Qualität (70 %), die Community-Hilfe bei Problemen (10 %) und die Verfügbarkeit von Plugins oder externen Tools (20 %).*

3.4 Vorgehen der prototypischen Umsetzung

Die praktische Erprobung erfolgt schrittweise. Zunächst wird für beide Frameworks eine lokale Testumgebung aufgesetzt, wobei notwendige Konfigurationsschritte und Workarounds dokumentiert werden. Anschließend werden die drei definierten Testszenarien nacheinander in Selenium und Cypress implementiert. Um eine Vergleichbarkeit

zu gewährleisten, wird für beide Werkzeuge derselbe Ausgangszustand der Anwendung genutzt.

3.5 Datenerhebung und Auswertung

Die Datenerhebung findet parallel zur prototypischen Umsetzung statt. Quantitative Daten (Dauer der Testausführung, Code-Struktur) und qualitative Daten (Hürden bei der Netzwerkintegration, Fehlermeldungen) werden strukturiert festgehalten.

Nach Abschluss der Implementierung werden die erhobenen Daten in das Kriterienmodell überführt. Jedes Framework erhält pro Kriterium eine Bewertung auf einer festen **Skala von 1 bis 10 Punkten**. Diese Punktzahl wird mit der zuvor definierten Gewichtung multipliziert. Die Summe aller gewichteten Punkte ergibt den Gesamtnutzwert.

4 Implementierung und Durchführung der Prototypen

Dieses Kapitel stellt die praktische Testumgebung, die ausgeführten Testszenarien sowie den technischen und syntaktischen Vergleich zwischen Selenium und Cypress dar. Die Ausformulierung stellt dabei konkrete Projektbezüge zur Webanwendung Puma bei der JUMO GmbH & Co. KG her.

4.1 Testumgebung und Rahmenbedingungen

Die Tests wurden im betrieblichen Kontext der JUMO GmbH & Co. KG durchgeführt. Untersuchungsgegenstand ist die interne Webanwendung *Puma*, die aus einem konkreten betrieblichen Bedarf heraus entstanden ist. Da die vorherige Verwaltung firmeninterner Projekte zunehmend unübersichtlich und schwer nachvollziehbar wurde, dient Puma nun als zentrale Lösung zur Projekt- und Kostenverwaltung. Ein Projekt entspricht in der Anwendung im Wesentlichen einem Arbeitsplan, der um eine detaillierte Kosten- und Stundenplanung erweitert wurde. Darüber hinaus bietet das System Funktionen gängiger Projektplanungstools: Es lassen sich Personen und Teams zuordnen, relevante Dokumente zentral hinterlegen und Projektdaten über eine Schnittstelle direkt mit dem SAP-System synchronisieren.

Das in Abbildung 1 dargestellte vereinfachte Komponentendiagramm veranschaulicht die fachliche und technische Struktur der Anwendung. Im Mittelpunkt steht die Projektverwaltung als zentrale Entität. Ein Projekt ist mit Modulen für Kosten- und Stundenplanungen, Ressourcen, Teams sowie der Dokumentenverwaltung verknüpft. Zur Sicherstellung der Nachvollziehbarkeit von Änderungen werden Aktivitäten, Kommentare, Benachrichtigungen und Historieneinträge erfasst. Die Suchfunktionen werden über Apache Solr abgebildet. Technische Basisdienste wie Logging, Caching, Fehlerbehandlung und allgemeine Hilfsklassen unterstützen die fachlichen Komponenten im Hintergrund.

Der Technologie-Stack der Anwendung basiert auf Java. Als Entwicklungsumgebung kam Eclipse zum Einsatz, während Maven für das Build-Management verwendet wurde. Die Bereitstellung der Anwendung für die lokale Testausführung erfolgte über einen lokalen Tomcat-Server. Der verwendete Port für die Testausführung ist **8082**. Ohne diesen laufenden Tomcat-Server ist keine Testausführung möglich, da die Webanwendung lokal ansonsten nicht erreichbar ist.

Die Ausführung der Tests unterlag den strengen Sicherheitsrichtlinien des Firmennetzwerks. Aufgrund der unternehmensinternen Proxy-Vorgaben war die Installation externer Komponenten (wie beispielsweise npm-Pakete für Node.js) nur eingeschränkt möglich.

Die Tests wurden lokal auf einem Entwickler-Laptop durchgeführt. Die Spezifikationen des Systems lauten wie folgt:

- **Betriebssystem:** Windows 11 Enterprise
- **Version:** 24H2

- **Installationsdatum:** 29.08.2025
- **Betriebssystembuild:** 26100.8457

Für die Testdurchführung wurde ausschließlich der Browser Microsoft Edge in der Version 120 verwendet.

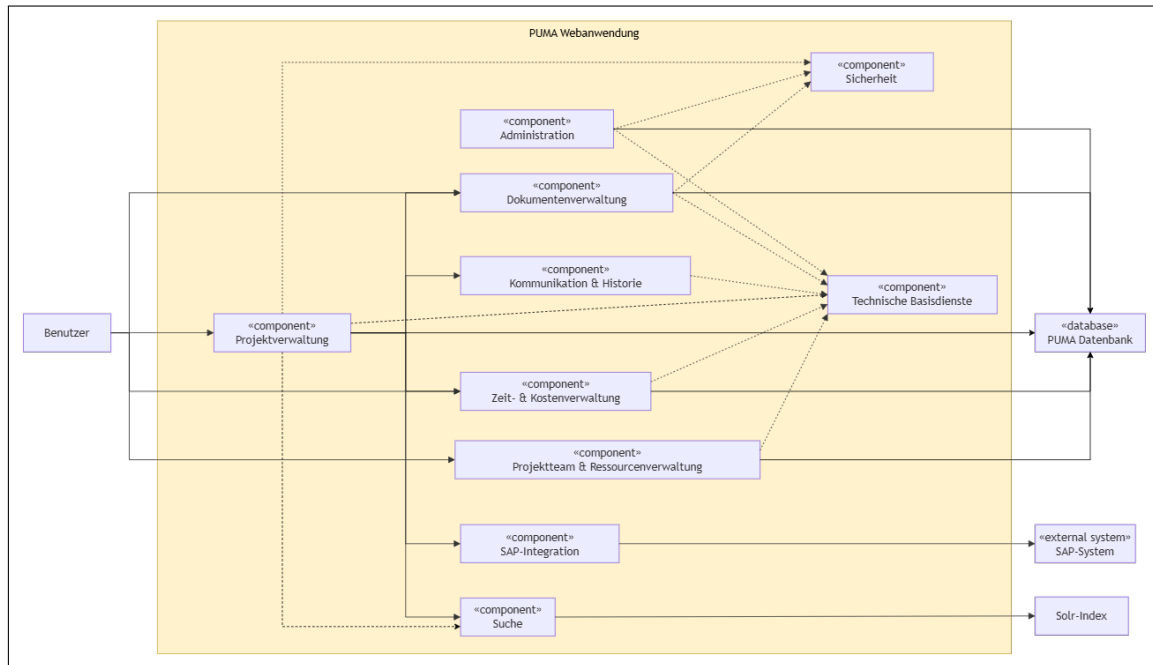


Abbildung 1: Fachliche und technische Struktur der Webanwendung Puma

4.2 Beschreibung der Testszenarien

Dieses Unterkapitel beschreibt die getesteten Anwendungsszenarien systematisch und ordnet diese den jeweiligen Testimplementierungen zu. Zur besseren Übersichtlichkeit wird im Folgenden auf die Angabe vollständiger Dateipfade verzichtet. Stattdessen werden die zugehörigen Testdateien direkt mit ihrem Dateinamen benannt. Die Namenskonvention orientiert sich an einer fortlaufenden Nummerierung der Szenarien (z. B. 01_ . . . für das erste Szenario). Die vollständigen Pfade und Quellcodes sind im digitalen Anhang (Git-Repository) zu finden.

4.2.1 Szenario 1: Login

Das Ziel dieses Tests ist es zu prüfen, ob sich ein Benutzer erfolgreich an der Puma-Webanwendung anmelden kann. Die Startbedingungen setzen voraus, dass Puma lokal über den Tomcat-Server auf Port 8082 erreichbar ist und ein gültiger Benutzer mit Benutzernamen und Passwort zur Verfügung steht.

Der Ablauf des Tests beginnt mit dem Öffnen der Login-Seite. Nach der Eingabe von Benutzernamen und Passwort wird der Login-Button betätigt. Anschließend wird geprüft, ob eine Weiterleitung auf die erwartete Zielseite erfolgt.

Das erwartete Ergebnis ist, dass der Benutzer erfolgreich angemeldet wird und der Redirect im Test validiert wird. Die automatisierte Umsetzung erfolgte in den Testdateien **01_PumaLoginTest.java** (Selenium) und **01_login-ui.cy.js** (Cypress).

4.2.2 Szenario 2: Neues Projekt bzw. neue Aktivität anlegen

In diesem Szenario wird geprüft, ob ein neues Projekt bzw. eine neue Aktivität korrekt über die Benutzeroberfläche angelegt werden kann. Als Startbedingung muss der Benutzer erfolgreich angemeldet sein. Die Anwendung muss lokal erreichbar sein und die benötigten Dropdown-Werte müssen im System zur Verfügung stehen.

Der Test navigiert zur Maske für die Projekt- bzw. Aktivitätsanlage. Das Stammbblatt wird durch Eingabe der benötigten Projektdaten ausgefüllt. Eine Besonderheit bilden hier Dropdowns mit **select2**, die dynamisch reagieren und daher gezielt im Code angesprochen werden müssen. Nach dem Ausfüllen wird das Formular gespeichert und die Systemrückmeldung geprüft.

Als erwartetes Ergebnis wird das Projekt oder die Aktivität erfolgreich angelegt. Die Anwendung zeigt eine korrekte Bestätigung an, und die Daten sind im System vorhanden. Die zugehörigen Testdateien sind **02_NewProjektTest.java** und **02_NewActivityAnlegenTest.java** für Selenium sowie **02_newProjekt.cy.js** und **02_newActivity.cy.js** für Cypress.

4.2.3 Szenario 3: Aktivität bearbeiten

Das Ziel des dritten Szenarios ist die Prüfung, ob eine bestehende Aktivität geöffnet, geändert, gespeichert und anschließend korrekt validiert werden kann. Die Startbedingungen erfordern, dass der Benutzer angemeldet ist und eine bestehende Aktivität aufgerufen werden kann.

Der Ablauf sieht die Navigation zur Aktivität sowie das Öffnen der Bearbeitungsansicht vor. Es folgt die Änderung definierter Werte (z. B. Fortschritt oder Datum). Nach dem Speichern wird die Aktivität erneut aufgerufen, um zu validieren, ob die Änderungen korrekt übernommen wurden.

Es wird erwartet, dass die Aktivität erfolgreich gespeichert wird und die geänderten Werte korrekt in der Oberfläche angezeigt werden. Die Umsetzung für dieses Szenario befindet sich in **03_EditActivityTest.java** und **03_ControlEditedActivityTest.java** (Selenium) sowie **03_editActivity.cy.js** und **03_controlEditedActivity.cy.js** (Cypress).

4.3 Setup und technische Integration

Dieses Unterkapitel erläutert, wie Selenium und Cypress technisch in die bestehende Entwicklungsumgebung integriert wurden und welche spezifischen Herausforderungen dabei aufgetreten sind.

4.3.1 Selenium

Selenium wurde als Bibliothek in die bestehende Java-Umgebung integriert. Die Einbindung erfolgte unkompliziert über die **pom.xml** mit Maven. Da im Projekt ohnehin mit Java, Eclipse und Maven gearbeitet wird, passt das Framework sehr gut zur bestehenden Entwicklungsumgebung. Die Integration verlief weitgehend reibungslos. Ein anfängliches Kompatibilitätsproblem mit der im Unternehmen verwendeten älteren JDK-Version konnte umgangen werden.

Selenium ließ sich aufgrund der bestehenden Java- und Maven-Struktur gut in die vorhandene Entwicklungsumgebung integrieren.

4.3.2 Cypress

Im Gegensatz zur etablierten Java-basierten Integration von Selenium setzt Cypress eine Node.js- und npm-Umgebung voraus. Im vorliegenden Unternehmenskontext erwies sich diese technologische Anforderung als erste Hürde: Strikte Proxy- und Sicherheitsrichtlinien blockierten die reguläre Paketinstallation über npm, weshalb eine dedizierte IT-Freigabe zwingend erforderlich war.

Zwar kann eine lokale Installation über ein ZIP-Archiv als kurzfristiger Workaround dienen, dieser Ansatz wurde jedoch verworfen. Er eignet sich nicht für wiederholbare, skalierbare Continuous-Integration-Prozesse (CI) und erschwert die saubere Versionsverwaltung. Folglich bleibt der initiale Einrichtungsaufwand für Cypress – trotz der grundsätzlichen Lösbarkeit durch administrative Freigabeprozesse – im direkten Vergleich zu Selenium organisatorisch und technisch umständlicher.

4.4 Syntax- und Architekturvergleich

Die Frameworks Selenium und Cypress unterscheiden sich grundlegend in Bezug auf Architektur, Syntax, Wartbarkeit und Synchronisation.

4.4.1 Architektur

Selenium wird als Bibliothek in die bestehende Java/JUnit-Umgebung integriert [1]. Es arbeitet als Bestandteil des vorhandenen Maven-Projekts, wodurch die Testautomatisierung eng mit der bestehenden Backend- bzw. Java-Struktur verbunden ist. Cypress hingegen ist ein eigenständiges Tooling auf Basis von Node.js und bringt eine eigene Testumgebung mit. Es läuft nicht als Java-Bibliothek, sondern agiert als separates End-to-End-Testframework.

Zusammenfassend: Selenium erweitert die vorhandene Java-Testumgebung, während Cypress eine eigenständige Testplattform auf Basis von Node.js bereitstellt.

4.4.2 Ausdrucksstärke der Syntax

Cypress zeichnet sich durch eine Syntax aus, die näher an natürlicher Sprache ist. Befehle lassen sich über das Konzept des Chainings gut lesbar aneinanderreihen. Durch Befehle

wie **cy.get(...).type(...)** entsteht eine kompakte und gut lesbare Testbeschreibung.

Selenium ist hingegen technischer und stärker objektorientiert geprägt. Bevor eine Interaktion stattfinden kann, müssen DOM-Elemente in der Regel explizit als **WebElement** gespeichert werden. Die tatsächliche Interaktion erfolgt erst im nächsten Schritt über Methoden wie **clear()** und **sendKeys()**.

4.4.3 Wartbarkeit

Bei der Nutzung von Selenium entstehen ohne strukturierende Maßnahmen schnell Wiederholungen im Quellcode. Um Redundanzen zu vermeiden, sind Hilfsmethoden oder die Anwendung des Page-Object-Patterns als notwendige Strukturierungsmaßnahmen unerlässlich [4].

Cypress ermöglicht durch Chaining und implizite Mechanismen häufig kompakteren Code. Gerade einfache Testabläufe wirken dadurch übersichtlicher, auch wenn bei steigender Projektkomplexität ebenso in Cypress eine strukturierte Testorganisation erforderlich sein kann.

4.4.4 Synchronisation

Ein zentraler Unterschied liegt in der Handhabung dynamischer Ladezeiten. In Selenium müssen dynamische Elemente häufig explizit mit Anweisungen wie **wait.until(...)** behandelt werden. Die Sichtbarkeit oder Klickbarkeit muss aktiv geprüft werden, was den Code umfangreicher macht.

Cypress verfügt über integrierte Auto-Waits. Viele Wartebedingungen sind bereits in den Basisbefehlen wie **cy.get()**, **type()** oder **click()** enthalten, wodurch sich der manuelle Synchronisationsaufwand maßgeblich verringert.

Der wesentliche Unterschied liegt darin, dass Selenium Wartebedingungen häufig explizit formuliert, während Cypress viele Synchronisationsaufgaben automatisch übernimmt.

4.4.5 Benutzeroberfläche und Test-Logging

Selenium bietet keine eigene Benutzeroberfläche, sondern wird typischerweise als klassischer JUnit-Test aus der IDE heraus gestartet. Zudem fehlt ein integriertes Logging der einzelnen Testschritte. Log-Ausgaben müssen manuell implementiert werden, andernfalls erhält man bei Fehlern oft nur komplexe Java-Stacktraces.

Cypress stellt dagegen einen interaktiven grafischen Test Runner bereit. Dieser beinhaltet ein automatisches *Command-Log*, das jede Interaktion in Echtzeit am Bildschirmrand protokolliert. Über eine integrierte Time-Travel-Funktion lässt sich der visuelle Zustand des Browsers zu jedem vergangenen Testschritt exakt prüfen, was manuellen Log-Aufwand vollständig überflüssig macht.

Es sei an dieser Stelle angemerkt, dass sich das fehlende visuelle Reporting und Logging in Selenium durch die Integration diverser Drittanbieter-Tools (wie beispielsweise All

lure Reports oder kommerziellen Cloud-Lösungen) aufrüsten ließe, um ein ähnliches Komfortniveau wie bei Cypress zu erreichen. In dieser Fallstudie werden solche Erweiterungen jedoch nicht berücksichtigt. Die Beschaffung, Lizenzierung und administrative Freigabe externer Tools ist im stark reglementierten Unternehmenskontext häufig mit zusätzlichen Kosten, Datenschutzprüfungen und langwierigen IT-Prozessen verbunden. Daher fokussiert sich dieser Vergleich bewusst auf die *Out-of-the-box*-Funktionalitäten der beiden Frameworks.

Zusammenfassend: Während Selenium in der Standardausführung auf manuelle Log-Implementierungen und IDE-Konsolen angewiesen ist, bietet Cypress durch seinen Test Runner und automatisches Logging von Haus aus eine deutlich bessere visuelle Nachvollziehbarkeit.

4.5 Code-Gegenüberstellung: Projektanlage

Anhand eines ausgewählten Codebeispiels wird im Folgenden der strukturelle Aufbau beider Frameworks bei typischen UI-Interaktionen gegenübergestellt. Der Schwerpunkt dieses Vergleichs liegt auf dem Befüllen von Formularfeldern, konkret dem Setzen des Projektnamens und des Enddatums bei der Projektanlage.

Wie in Listing 1 erkennbar, wird für jedes relevante Element in Selenium zunächst ein explizites Warten (Wait) definiert. Beim Feld `project_name` wird auf die Sichtbarkeit gewartet, beim Feld `project_endDate` auf die Klickbarkeit. Erst danach können die Felder geleert und mit den Testdaten befüllt werden. Dies macht den Code zwar robust gegenüber dynamischen Ladezeiten, erhöht jedoch die Zeilenanzahl und verleiht dem Test einen stark technischen Charakter.

Cypress nutzt im Gegensatz dazu, wie in Listing 2 dargestellt, die Anweisung `cy.get()` zur Auswahl des Elements. Notwendige Wartebedingungen sind in den Cypress-Standardbefehlen bereits implizit integriert, sodass die Aktionen direkt verkettet (Chaining) werden können. Für das Datumsfeld wird zusätzlich `scrollIntoView()` verwendet, um sicherzustellen, dass sich das Element im sichtbaren Viewport befindet.

Aus dieser Gegenüberstellung lassen sich folgende Bewertungsaspekte für die Nutzwertanalyse ableiten:

1. **Synchronisation:** Selenium erfordert bei dynamischen Oberflächen die manuelle Definition expliziter Waits. Cypress kapselt diese Wartebedingungen in seinen Standardbefehlen.
2. **Codeumfang:** Selenium benötigt mehr Programmzeilen, da Wartebedingungen und Elementvariablen separiert werden. Cypress erreicht durch das Chaining eine kompaktere Darstellung.
3. **Lesbarkeit:** Der Selenium-Code ist stark technisch strukturiert. Die Cypress-Syntax liest sich hingegen eher wie eine direkte funktionale Beschreibung der Anwenderaktion.

```

1 // Projektname setzen + prüfen
2 WebElement projektname = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id(
3     "project_name")));
4 assertNotNull(projektname, "Feld 'Projektname' wurde nicht gefunden.");
5 projektname.clear();
6 projektname.sendKeys(PROJEKTNAME);
7 assertEquals(PROJEKTNAME, projektname.getAttribute("value"),
8     "Projektname wurde nicht korrekt gesetzt.");
9 // Bereich
10 selectDropdownByTextAndAssert("project_divisionId", "Informationstechnologie (IT)");
11
12 // Projektprofil
13 selectDropdownByTextAndAssert("project_sapProjectProfile", "ZEW");
14
15 // Profitcenter
16 selectDropdownByTextAndAssert("project_sapProfitCenter", "CC10100400 - Ausbildung");
17
18 // Enddatum
19 WebElement endDate = wait.until(ExpectedConditions.elementToBeClickable(By.id("
20     project_endDate")));
21 assertNotNull(endDate, "Feld 'Enddatum' wurde nicht gefunden.");
22 endDate.clear();
23 endDate.sendKeys("17.08.2026");
24 assertEquals("17.08.2026", endDate.getAttribute("value"),
25     "Enddatum wurde nicht korrekt gesetzt.");

```

Quellcode 1: Projektanlage mit Selenium

```

1 // Neues Projekt anlegen + Aktivität
2 // Stammbblatt ausfüllen (Pflichtfelder):
3
4 // Projektname
5 cy.get('#project_name').type('Cypress Testprojekt')
6
7 // Bereich
8 cy.get('#project_divisionId')
9     .scrollIntoView()
10    .select('Informationstechnologie (IT)', { force: true })
11
12 // Projektprofil
13 cy.get('#project_sapProjectProfile')
14    .select('ZEW', { force: true })
15
16 // Profitcenter
17 cy.get('#project_sapProfitCenter')
18    .select('CC10100400 - Ausbildung', { force: true })
19
20 // Datum(Zeitspanne)
21 //Von
22 /*
23 cy.get('#project_startDate')
24    .clear()
25    .type('dd.mm.yyyy')
26 */
27
28 //Bis
29 cy.get('#project_endDate')
30    .scrollIntoView()
31    .clear()
32    .type('17.08.2026')

```

Quellcode 2: Projektanlage in Cypress

4. **Wartbarkeit:** Während Cypress einfache UI-Abläufe nativ kompakt hält, erfordert Selenium bei wachsendem Testumfang zwingend den Einsatz von Hilfsmethoden oder Architekturmustern (wie dem Page Object Model), um die Wartbarkeit sicherzustellen.

Zusammenfassend zeigt der Vergleich, dass Selenium eine sehr feingranulare Steuerung der Synchronisation ermöglicht, was jedoch den Code-Overhead erhöht. Cypress bietet insbesondere bei formularbasierten Interaktionen eine lesbarere und kompaktere Syntax, was die Einarbeitung und Pflege der Tests erleichtert.

5 Evaluation und Nutzwertanalyse

Nachdem im vorherigen Kapitel die prototypische Umsetzung der Testszenarien beschrieben wurde, erfolgt in diesem Kapitel die systematische Auswertung. Ziel ist es, die erhobenen Daten zu strukturieren und eine nachvollziehbare Basis für die Beantwortung der Forschungsfrage zu schaffen.

5.1 Durchführung und Testergebnisse

Im Rahmen der praktischen Evaluation wurden die ausgewählten regressionskritischen Testszenarien iterativ ausgeführt, um Daten hinsichtlich der Ausführungsgeschwindigkeit und der Zuverlässigkeit (Stabilität) zu erheben.

Die Ausführungsgeschwindigkeiten beider Frameworks zeigten beim initialen Testlauf identische Werte von jeweils circa 45 Sekunden (inklusive Start der Testumgebung und Browser-Initialisierung). Bei den darauffolgenden Folgeläufen reduzierte sich die Ausführungszeit bei beiden Tools auf eine Spanne von 22 bis 32 Sekunden. Cypress schließt beim vollständigen Projekttest minimal schneller ab, was darauf zurückzuführen ist, dass Aktionen direkter im Browser ausgeführt werden, während Selenium über den WebDriver mit expliziten Wartebedingungen arbeitet.

Zur Bewertung der Unzuverlässigkeit (*Flakiness*) wurden die Szenarien für den *Login* sowie das *Ändern von Datensätzen* auf 50 Durchläufe skaliert. Die Tendenz war signifikant: Bei Cypress traten in 4 von 50 Durchläufen Probleme auf, während Selenium mit 11 von 50 Durchläufen eine deutlich höhere Fehlerquote verzeichnete. Selenium reagiert wesentlich empfindlicher auf Ladezeiten dynamischer Ajax-Elemente, wohingegen Cypress durch integrierte Retry-Mechanismen robuster arbeitet.

5.2 Auswertung des Kriterienmodells

Die strukturierte Auswertung basiert auf der Bewertungsmatrix nach Kühnapfel [5]. Das Modell umfasst fünf Hauptkriterien, die sich jeweils in gewichtete Unterkriterien aufteilen. Für jedes Unterkriterium i wird eine Bewertung auf einer Skala von 1 bis 10 vergeben. Diese Teilbewertung (B_i) wird mit der prozentualen Gewichtung innerhalb der Kategorie (g_i) multipliziert und anschließend mit der Gewichtung der Hauptkategorie (G_j) verrechnet, um den finalen Kategoriewert (K_j) zu erhalten:

$$K_j = \left(\sum_{i=1}^n B_i \cdot g_i \right) \cdot G_j \quad (1)$$

Im Folgenden werden die Punktevergaben für sämtliche Unterkriterien detailliert begründet und die resultierenden Teilnutzwerte transparent berechnet.

5.2.1 Integration in die Infrastruktur (Gewichtung: 25 %)

Dieses Kriterium bewertet, wie nahtlos sich das Tool in die bestehende Java-/Eclipse-Umgebung bei JUMO integrieren lässt.

- **Projekt-Anlage & Tool-Start (40 %):** Selenium (8 Punkte) ließ sich problemlos über die `pom.xml` in den Maven-Build integrieren. Cypress (5 Punkte) benötigt eine separate Node.js-Umgebung, deren `npm install`-Prozess zunächst durch firmeninterne Proxys blockiert wurde und IT-Freigaben erforderte.
- **Debugging & Ausführen (30 %):** Cypress (9 Punkte) punktet durch den grafischen Test Runner und die Time-Travel-Funktion, die Fehler sofort visuell nachvollziehbar machen. Selenium (6 Punkte) stützt sich auf klassisches IDE-Debugging, was funktional, aber visuell weniger intuitiv ist.
- **CI-/Reporting-Anschlussfähigkeit (30 %):** Beide Tools (jeweils 7 Punkte) lassen sich gut in Pipelines integrieren, wobei Selenium naturgemäß besser zum bestehenden Java-Standard-Workflow passt.

Berechnung des Teilnutzwerts:

Selenium: $((8 \cdot 0,40) + (6 \cdot 0,30) + (7 \cdot 0,30)) \cdot 0,25 = 7,10 \cdot 0,25 = 1,775$

Cypress: $((5 \cdot 0,40) + (9 \cdot 0,30) + (7 \cdot 0,30)) \cdot 0,25 = 6,80 \cdot 0,25 = 1,700$

5.2.2 Lernkurve und Syntax (Gewichtung: 20 %)

Hier wird bewertet, wie schnell das Team produktiv wird und wie lesbar sowie wartbar der geschriebene Testcode ist.

- **Einstieg / erster Test (40 %):** Cypress (9 Punkte) ermöglicht durch sprechende Befehle einen extrem schnellen Start. Selenium (7 Punkte) setzt solides Java-Verständnis und Boilerplate-Code voraus.
- **Usability (10 %):** Cypress (9 Punkte) fühlt sich durch das *Chaining* sehr intuitiv und nah am manuellen Test an. Selenium (6 Punkte) erfordert mehr technische Zwischenschritte zur Element-Instanziierung.
- **Lesbarkeit & Wartbarkeit (25 %):** Die Cypress-Syntax (8 Punkte) bleibt auch bei längeren Tests gut verständlich. Selenium (7 Punkte) erfordert konsequent die Nutzung von Patterns (wie Page Objects), um nicht unübersichtlich zu werden.
- **Fehlermeldungen / Diagnose (25 %):** Cypress (9 Punkte) zeigt die genaue Ursache direkt im DOM an. Selenium (6 Punkte) wirft oftmals Stacktraces aus, deren Fehlerursache schwerer zu lokalisieren ist.

Berechnung des Teilnutzwerts:

Selenium: $((7 \cdot 0,40) + (6 \cdot 0,10) + (7 \cdot 0,25) + (6 \cdot 0,25)) \cdot 0,20 = 6,65 \cdot 0,20 = 1,330$

$$\text{Cypress: } ((9 \cdot 0,40) + (9 \cdot 0,10) + (8 \cdot 0,25) + (9 \cdot 0,25)) \cdot 0,20 = 8,75 \cdot 0,20 = 1,750$$

5.2.3 Test-Ausführung (Gewichtung: 20 %)

Diese Kategorie bewertet die Geschwindigkeit im Entwicklungsalltag und bei potenzieller Skalierung.

- **Geschwindigkeit lokal (60 %):** Cypress (8 Punkte) lief bei der Projekterstellung lokal flüssiger und direkter. Selenium (7 Punkte) wies durch den WebDriver-Overhead eine minimal langsamere Ausführung auf.
- **Geschwindigkeit in CI / Skalierung (40 %):** Beide Frameworks (jeweils 7 Punkte) bieten die Möglichkeit zur parallelen und Headless-Ausführung, wofür bei großen Suites eine entsprechende Infrastruktur bereitgestellt werden muss.

Berechnung des Teilnutzwerts:

$$\text{Selenium: } ((7 \cdot 0,60) + (7 \cdot 0,40)) \cdot 0,20 = 7,00 \cdot 0,20 = 1,400$$

$$\text{Cypress: } ((8 \cdot 0,60) + (7 \cdot 0,40)) \cdot 0,20 = 7,60 \cdot 0,20 = 1,520$$

5.2.4 Stabilität (Gewichtung: 20 %)

Die Zuverlässigkeit der Ergebnisse und das Abfangen von Timing-Problemen sind kritisch für die Akzeptanz im Team.

- **Warten & Synchronisation (40 %):** Cypress (8 Punkte) nutzt ein integriertes Auto-Waiting für dynamische Seiten. Selenium (6 Punkte) erfordert bei asynchronen Oberflächen stetig explizite Eingriffe (z. B. `wait.until(...)`).
- **Zuverlässigkeit über mehrere Läufe (55 %):** Cypress (9 Punkte) bewies im Stresstest eine sehr niedrige Fehlerrate. Selenium (6 Punkte) schlug ohne Code-Änderungen signifikant häufiger durch Timing-Probleme fehl.
- **Debug-Hilfen bei Fehlern (5 %):** Cypress (9 Punkte) visualisiert Fehlzustände durch Snapshots perfekt. Selenium (6 Punkte) ist hier primär auf Trace-Logs angewiesen.

Berechnung des Teilnutzwerts:

$$\text{Selenium: } ((6 \cdot 0,40) + (6 \cdot 0,55) + (6 \cdot 0,05)) \cdot 0,20 = 6,00 \cdot 0,20 = 1,200$$

$$\text{Cypress: } ((8 \cdot 0,40) + (9 \cdot 0,55) + (9 \cdot 0,05)) \cdot 0,20 = 8,60 \cdot 0,20 = 1,720$$

5.2.5 Dokumentation und Community (Gewichtung: 15 %)

Hier wird geprüft, wie gut die Frameworks durch offizielle Webseiten und die Community unterstützt werden.

- **Doku-Qualität (70 %):** Die Cypress-Dokumentation (9 Punkte) ist herausragend, aktuell und stark auf konkrete Use-Cases fokussiert. Selenium (8 Punkte) bietet ebenfalls eine sehr gute, wenn auch teils technischere und generischere Dokumentation.
- **Hilfe durch Community (10 %):** Für beide Tools (jeweils 8 Punkte) finden sich schnell Lösungsansätze für blockierende Probleme.
- **Ökosystem / Tools (20 %):** Beide (jeweils 8 Punkte) bieten eine breite Palette an Plugins und Integrationen für Reporter und CI-Systeme.

Berechnung des Teilnutzwerts:

Selenium: $((8 \cdot 0,70) + (8 \cdot 0,10) + (8 \cdot 0,20)) \cdot 0,15 = 8,00 \cdot 0,15 = \mathbf{1,200}$

Cypress: $((9 \cdot 0,70) + (8 \cdot 0,10) + (8 \cdot 0,20)) \cdot 0,15 = 8,70 \cdot 0,15 = \mathbf{1,305}$

5.3 Nutzwertanalyse und Gesamtauswertung

Der finale Gesamtnutzwert (N) ergibt sich aus der Addition der zuvor berechneten Kategoriewerte. Tabelle 1 fasst die exakten Ergebnisse der Verrechnung zusammen.

Tabelle 1: Ergebnis der Nutzwertanalyse gemäß Bewertungsmatrix

Kriterium	Gewichtung	Selenium (gew.)	Cypress (gew.)
Integration in die Infrastruktur	25 %	1,775	1,700
Lernkurve und Syntax	20 %	1,330	1,750
Test-Ausführung	20 %	1,400	1,520
Stabilität	20 %	1,200	1,720
Dokumentation und Community	15 %	1,200	1,305
Gesamtnutzwert	100 %	6,905	7,995

Das Fazit der Nutzwertberechnung zeigt ein klares Bild: Cypress ist mit einem Gesamtnutzwert von **7,995** aus rein entwicklungstechnischer Sicht (fachlich, syntaktisch und hinsichtlich der Teststabilität) dem Framework Selenium (**6,905**) überlegen. Einer sofortigen und reibungslosen Einführung im JUMO-Netzwerk werden jedoch aktuell durch die infrastrukturellen Rahmenbedingungen (erforderliche Node.js-Umgebung, Proxy-Restriktionen) gravierende Hürden gesetzt.

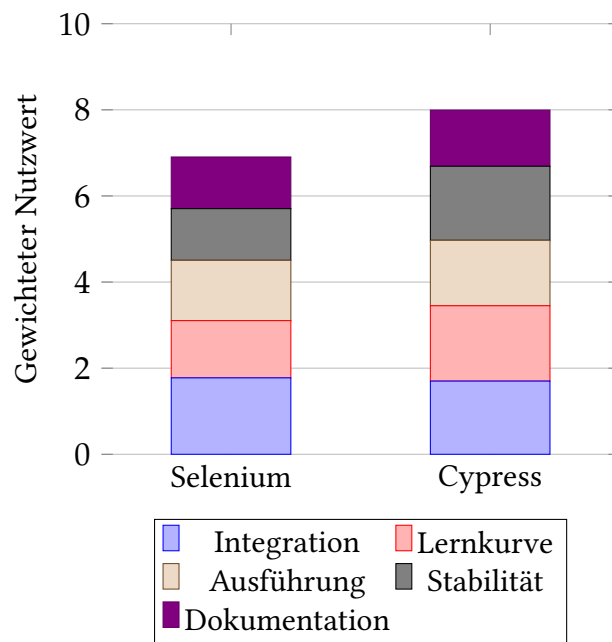


Abbildung 2: Grafische Gegenüberstellung der Teil- und Gesamtnutzwerte

6 Fazit und Ausblick

6.1 Beantwortung der Forschungsfrage und Handlungsempfehlung

Ziel dieser Fallstudie war es zu evaluieren, welches End-to-End-Test-Framework – Selenium oder Cypress – sich besser für die Automatisierung regressionskritischer Testszenarien im Umfeld der JUMO GmbH & Co. KG eignet. Die durchgeführte Nutzwertanalyse liefert hierauf eine differenzierte Antwort.

Aus rein technischer und entwicklungsorientierter Perspektive erweist sich **Cypress** als das klar überlegene Tool. Es punktet durch eine intuitive, leicht erlernbare Syntax und eine signifikant höhere Stabilität bei dynamischen Webanwendungen. Die integrierten Retry-Mechanismen reduzieren die Fehleranfälligkeit bei asynchronen Ladezeiten drastisch. Zudem bietet Cypress durch seinen interaktiven Test Runner und das automatische Command-Log eine überragende Developer Experience, da Fehlerursachen visuell nachvollzogen werden können, ohne manuelles Logging implementieren zu müssen.

Dennoch führt diese technische Überlegenheit nicht zu einer uneingeschränkten Empfehlung. Die infrastrukturellen Voraussetzungen der JUMO GmbH & Co. KG basieren stark auf einem etablierten Java-Ökosystem, während Cypress zwingend eine Node.js-Umgebung erfordert. Die in der Evaluierung aufgetretenen Konflikte mit den strengen Proxy- und Sicherheitsrichtlinien des Unternehmens bremsen eine nahtlose Integration und damit den Automatisierungsgewinn derzeit massiv aus.

Selenium hingegen stellt unter den aktuellen Rahmenbedingungen die pragmatischere Lösung dar, da es als Java-Bibliothek direkt und ohne administrativen Zusatzaufwand in das bestehende Maven-Projekt integriert werden kann. Diese nahtlose technische Integration wird jedoch mit erheblichen Nachteilen im Entwicklungsalltag erkauft: Selenium erfordert eine spürbar steilere Lernkurve, generiert durch notwendige explizite Wartebedingungen und Element-Instanziierungen einen hohen Code-Overhead (Boilerplate-Code) und zwingt Entwickler dazu, Testschritte manuell zu protokollieren. Dies erschwert langfristig die Wartbarkeit der Testsuite und bremst die Einarbeitung neuer Teammitglieder.

Die **Handlungsempfehlung** für das Unternehmen lautet daher: Kurzfristig sollte Selenium für die Automatisierung der regressionskritischen Tests der Puma-Anwendung genutzt werden, da die Integration sofort umsetzbar ist und somit erste Testaufwände zeitnah reduziert werden können. Um langfristig dem hohen Code-Overhead von Selenium entgegenzuwirken, ist dabei zwingend der Einsatz strukturierender Entwurfsmuster wie dem Page Object Model zu empfehlen. Mittelfristig sollte jedoch evaluiert werden, ob die IT-Infrastruktur (insbesondere Proxy-Freigaben und CI/CD-Pipelines) für Node.js-Umgebungen geöffnet werden kann. Ein späterer Wechsel auf Cypress ist stark anzuraten, um von der überlegenen Teststabilität, der flacheren Lernkurve und den weitaus besseren Debugging-Werkzeugen zu profitieren.

6.2 Kritische Reflexion und Methodengrenzen

Die in dieser Fallstudie angewandte Methodik und die daraus abgeleiteten Ergebnisse unterliegen gewissen Einschränkungen, die bei der abschließenden Bewertung zu berücksichtigen sind.

Zunächst basiert die Bewertung der CI- und Reporting-Integration für beide Tools teilweise auf theoretischen Annahmen. Im Rahmen des zeitlich begrenzten Praxisprojekts lag der Fokus auf der prototypischen Erstellung lokaler Tests; eine vollständige Integration in eine automatisierte Build-Pipeline wurde nicht abschließend praktisch umgesetzt. Des Weiteren ist zu beachten, dass qualitative Bewertungskriterien wie *Lesbarkeit der Syntax* und *Usability des Test Runners* zwangsläufig einer subjektiven Wahrnehmung des Evaluierenden unterliegen, auch wenn sie durch konkrete Implementierungserfahrungen untermauert wurden.

Abschließend ist eine klare fachliche Differenzierung bezüglich der identifizierten Schwächen von Cypress notwendig: Die gravierendsten Punktabzüge, welche bei der Projekterstellung und Installation auftraten, stellen keine originären Schwächen des Tools dar. Sie resultieren vielmehr maßgeblich aus den rigiden Sicherheits- und Proxy-Richtlinien der Unternehmensinfrastruktur. Würde das Unternehmen standardmäßig Node.js-Umgebungen und entsprechende Paket-Freigaben unterstützen, fiel die Nutzwertanalyse noch deutlicher zugunsten von Cypress aus.

6.3 Ausblick

Zukünftige Arbeiten könnten die in dieser Fallstudie identifizierten Einschränkungen aufgreifen. Eine logische Fortführung wäre die praktische Implementierung beider Frameworks in einer vollständigen CI/CD-Pipeline (z. B. Jenkins oder GitLab CI), um die theoretischen Annahmen zur Reporting- und Ausführungsfähigkeit im automatisierten Build-Prozess empirisch zu belegen.

Darüber hinaus sollte das Unternehmen die strategische Entscheidung treffen, inwiefern moderne JavaScript-/TypeScript-basierte Testing-Tools zukünftig gefördert werden sollen, um langfristig die Effizienz in der Qualitätssicherung von Webanwendungen zu steigern.

Literaturverzeichnis

- [1] A. Spillner und T. Linz, *Basiswissen Softwaretest: Aus-und Weiterbildung zum Certified Tester–Foundation Level nach ISTQB-Standard*. dpunkt. verlag, 2024.
- [2] Selenium Project, *Selenium Documentation*, <https://www.selenium.dev/documentation/>, [Online; abgerufen am 13.05.2026], 2024.
- [3] Cypress.io, *Cypress Documentation*, <https://docs.cypress.io/>, [Online; abgerufen am 13.05.2026], 2024.
- [4] P. Liggesmeyer, *Software-Qualität: Testen, Analysieren und Verifizieren von Software*, 2. Spektrum Akademischer Verlag, 2009.
- [5] J. B. Kühnapfel, *Scoring und Nutzwertanalysen: Ein Leitfaden für die Praxis*. Springer, 2021.

Glossar

Auto-Waiting Ein Mechanismus, bei dem das Test-Framework automatisch auf den Zustand von DOM-Elementen wartet, bevor eine Aktion ausgeführt wird.. 1

Boilerplate-Code Wiederkehrende Code-Fragmente, die mit wenig oder gar keiner Änderung an mehreren Stellen im Testcode wiederverwendet werden müssen.. 1

Chaining Eine Programmiertechnik, bei der mehrere Methodenaufrufe direkt aneinandergereiht werden, was zu einer kompakteren Syntax führt.. 1

Continuous Integration (CI) Fortlaufende Integration von Code-Änderungen in ein gemeinsames Repository, oft begleitet von automatisierten Build- und Testprozessen.. 1

Document Object Model (DOM) Eine Programmierschnittstelle für HTML-Dokumente, die die Struktur der Webseite als Baum repräsentiert.. 1

End-to-End-Test (E2E-Test) Ein Testverfahren, das das gesamte Softwaresystem über alle Schichten hinweg unter realitätsnahen Bedingungen aus Sicht des Endanwenders prüft.. 1

Flaky Test Ein unzuverlässiger automatisierter Test, der bei gleichem Systemzustand inkonsistente Ergebnisse (Erfolg oder Fehlschlag) liefert.. 1

Nutzwertanalyse (NWA) Ein methodisches Verfahren zur systematischen Bewertung und zum Vergleich mehrerer Handlungsalternativen anhand gewichteter Kriterien.. 1

Page Object Model (POM) Ein Entwurfsmuster in der Testautomatisierung zur Kapselung der Benutzeroberfläche in Klassen, um Redundanzen zu minimieren und die Wartbarkeit zu erhöhen.. 1

Regressionstest Ein Test zur Überprüfung, ob bestehende Funktionalitäten der Software nach Änderungen weiterhin fehlerfrei arbeiten.. 1

Time-Travel-Funktion Ein Feature (z. B. in Cypress), das den exakten visuellen Zustand der Weboberfläche zu jedem vergangenen Testschritt speichert und nachträglich abrufbar macht.. 1

WebDriver Ein Standard-Protokoll zur nativen Fernsteuerung von Webbrowsern, das primär von Selenium genutzt wird.. 1

Abkürzungsverzeichnis

API Application Programming Interface. 1

CD Continuous Delivery / Deployment. 1

E2E End-to-End. 1

IDE Integrated Development Environment. 1

IT Informationstechnologie. 1

JDK Java Development Kit. 1

SAP Systeme, Anwendungen und Produkte in der Datenverarbeitung. 1

UI User Interface. 1

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Alle sinngemäß und wörtlich übernommenen Textstellen aus fremden Quellen wurden kenntlich gemacht.

Fulda, den 17.08.2026

Marcel Licht

A Erklärung zur Nutzung von KI-Werkzeugen

Bei der Erstellung dieses Exposés wurden KI-Werkzeuge unterstützend genutzt:

Verwendete Tools: Google Gemini 3.1 Pro

- Hilfe bei der sprachlichen Überarbeitung und Verbesserung eigener Texte (Formulierungshilfe, Grammatik, und Rechtschreibprüfung)
- Hilfe bei LaTeX Befehlen und Formatierungen (z.B. Quellen in der references.bib oder Tabelle)
- Überprüfung auf Vollständigkeit der Anforderungen
- Unterstützung bei der Literaturrecherche

B Bewertungsmatrix (Excel)

In diesem Abschnitt ist die vollständige Bewertungsmatrix der Nutzwertanalyse abgebildet, die als Grundlage für die finale Auswertung diene.

Kriterium	Beschreibung	Gewichtung	Selenium (Score 1-10)	Cypress (Score 1-10)
Integartion (Java/ Eclipse)	Bewertet wird, wie nahtlos das Tool in unsere Java-/Eclipse-Umgebung integrierbar ist: Setup in Eclipse, Debugging, Build-Integration (Maven/Gradle), Test-Runner (z. B. JUnit/TestNG), CI-Ausführung, Reporting und Pflegeaufwand. Hoher Score = wenig Zusatztools/Workarounds, stabiler Standard-Workflow in IDE und CI.	25%	1,775	1,7
Projekt-Anlage & Tool-Start in deinem Workflow	Wie einfach ist das setp zum testen zu integrieren	40%	8	5
Debugging & Ausführen	Wie gut kannst du Tests starten/stoppen, Fehler nachvollziehen	30%	6	9
CI/Reporting Anschlussfähigkeit	Wie gut lässt es sich später in Pipeline + Reports integrieren	30%	7	7
Lenkurve & Syntax	Bewertet wird, wie schnell das Team produktiv wird und wie gut lesbar/wartbar die Syntax ist: Verständlichkeit der API, Boilerplate-Anteil, Konsistenz, typische Patterns (z. B. Page Objects), Qualität der Fehlermeldungen und wie leicht neue Teammitglieder gute Tests schreiben können. Hoher Score = schnelle Einarbeitung + klare, wartbare Test-Codestruktur.	20%	1,33	1,75
Einstieg / erster Test	Wie schnell verstehst du die Basics und bekommst einen Test hin?	40%	7	9
Usability	Wie intuitiv findet man sich ein?	10%	6	9
Lesbarkeit & Wartbarkeit	Kannst du den Test nach 2 Wochen noch leicht verstehen/ändern?	25%	7	8
Fehlermeldungen / Diagnose	Wie klar sind Errors? Wie schnell findest du die Ursache?	25%	6	9
Test-Ausführung	Bewertet wird die Geschwindigkeit im Alltag: Start-/Setup-Zeit, Laufzeit einzelner Tests und Suiten, Parallelisierung, Ressourcenverbrauch (CPU/RAM) sowie Skalierung in CI. Hoher Score = schnelle Feedback-Zyklen (lokal & CI) und effiziente Parallel-Ausführung ohne übermäßigen Infrastrukturbedarf.	20%	1,4	1,52
Geschwindigkeit lokal	Wie schnell laufen Tests beim Entwickeln?	60%	7	8
Geschwindigkeit in CI / Skalierung	Bleibt es schnell, wenn es mehr Tests werden / parallel läuft?	40%	7	7
Stabilität	Bewertet wird die Zuverlässigkeit der Testergebnisse: wie gut das Tool Timing/Asynchronität abfängt (Waits/Auto-Waiting), Robustheit gegen UI-Änderungen, Stabilität in CI, sowie Diagnosefähigkeit (Logs/Traces/Screenshots) zur schnellen Fehleranalyse. Hoher Score = reproduzierbare Ergebnisse und niedrige Flaky-Rate.	20%	1,2	1,72
Warten & Synchronisation	Verhalten mit dynamischen Seiten klar, ohne viele Sleeps	40%	6	8
Zuverlässigkeit über mehrere Läufe	Wie oft "random" Fail ohne Codeänderung	55%	6	9
Debug-Hilfen bei Fehlern	Screenshots/Video/Trace/Logs (hilfreich?)	5%	6	9
Dokumentation & Community	Bewertet wird die Unterstützung durch Doku und Ökosystem: Qualität/Aktualität der Dokumentation, Umfang an Beispielen, Community-Aktivität, Problemlösung (Issues/Foren), Release-/Wartungsniveau und verfügbare Integrationen/Plugins. Hoher Score = schnell Lösungen finden und langfristig gut wartbar.	15%	1,2	1,305
Doku-Qualität	verständlich? Gibt es aktuell, gute Beispiele	70%	8	9
Hilfe (Community)	schnelle Lösung bei Stagnation	10%	8	8
Ökosystem/Tools	Reporter, Plugins, Integrationen	20%	8	8
Summe (Gesamtnutzwert)			6,905	7,995

Abbildung 3: Bewertungsmatrix der Nutzwertanalyse (Selenium vs. Cypress) – Teil 1

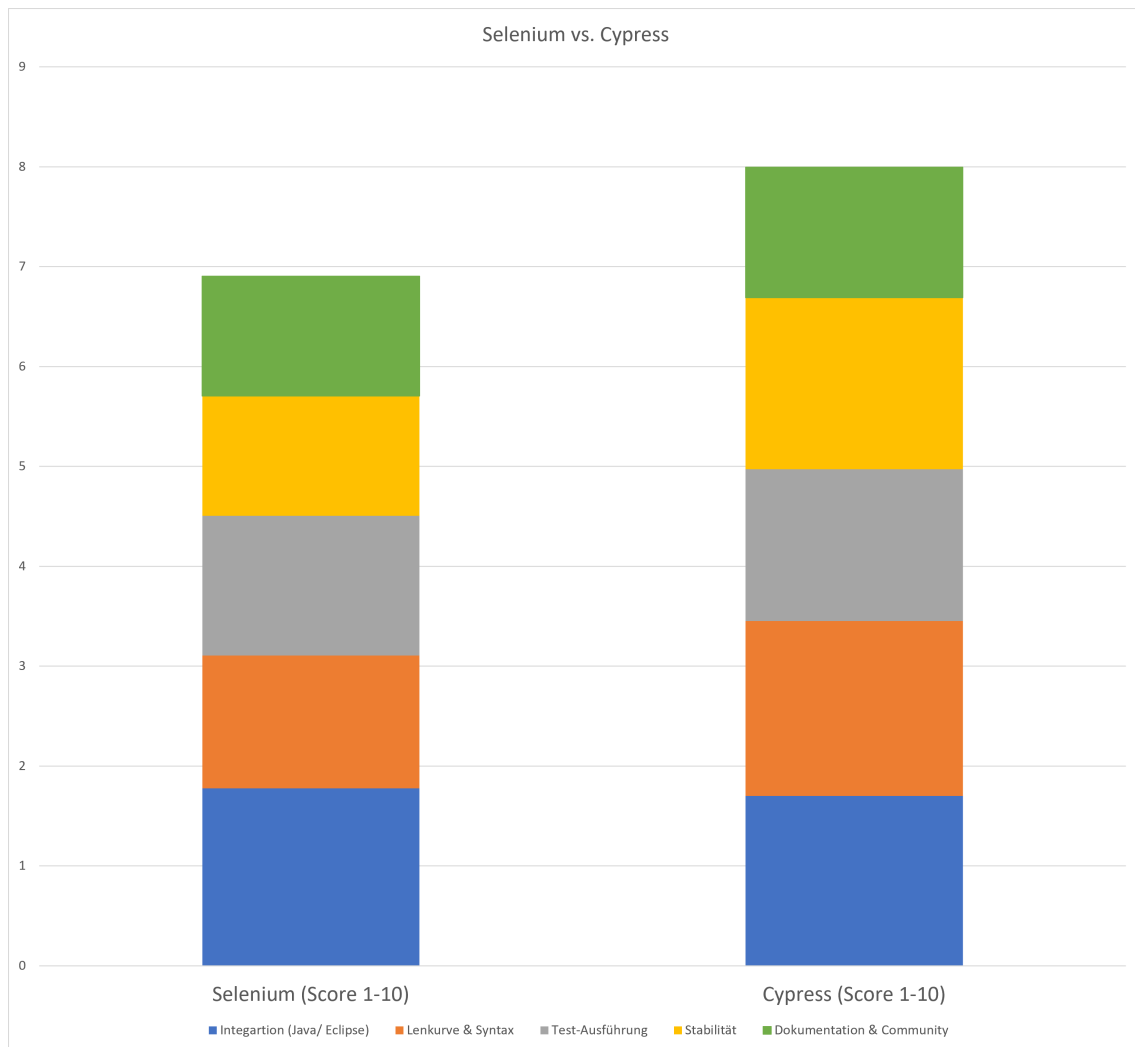


Abbildung 4: Bewertungsmatrix der Nutzwertanalyse (Selenium vs. Cypress) – Teil 2

C Testprotokolle (Testlogs)

In diesem Abschnitt sind die detaillierten Testprotokolle der automatisierten Testdurchläufe für Selenium und Cypress dokumentiert. Sie umfassen sowohl erfolgreiche Durchläufe als auch exemplarische Fehlerausgaben, die im Rahmen der Prototyp-Evaluation protokolliert wurden. Dieser Abschnitt dient dem Nachweis der Systemstabilität, der Transparenz bei Fehlerdiagnosen und unterstreicht die praktischen Ergebnisse der Nutzwertanalyse.

C.1 Testprotokolle: Selenium

```
1 [SELENIUM] Login-Test gestartet
2 [SELENIUM] Login-Seite geöffnet
3 [SELENIUM] Passwort eingegeben
4 [SELENIUM] Login-Button geklickt
5 [SELENIUM] Home-Seite wurde erreicht
6 [SELENIUM] Login erfolgreich geprüft
```

Quellcode 3: Selenium – Login (Erfolgreich)

```
1 _02_NewProjektTest
2 ui.pumaTests.new_Projekt._02_NewProjektTest
3 sollteNeuesProjektErstellen(ui.pumaTests.new_Projekt._02_NewProjektTest)
4 org.openqa.selenium.TimeoutException: Expected condition failed: waiting for element
   found by By.cssSelector: #project-dropdown-menu a[href='createProject'] to be
   clickable, but the element was not found: org.openqa.selenium.NoSuchElementException
   : no such element: Unable to locate element: {"method":"css selector","selector":"#
   project-dropdown-menu a[href='createProject']"}
5 (tried for 10 seconds with 500 milliseconds interval)
6 Build info: version: '4.43.0', revision: 'dd0f534'
7 System info: os.name: 'Windows 11', os.arch: 'amd64', os.version: '10.0', java.version:
   '25.0.2'
8 Driver info: org.openqa.selenium.edge.EdgeDriver
9 Capabilities {acceptInsecureCerts: false, browserName: MicrosoftEdge, browserVersion:
   151.0.4129.72, fedcm:accounts: true, goog:processID: 33636, ms:edgeOptions: {
   debuggerAddress: localhost:49771}, msedge: {msedgedriverVersion: 151.0.4129.78 (
   dd5b3842e0c9..., userDataDir: C:\Users\MARCEL~1.LIC\AppData...,
   networkConnectionEnabled: false, pageLoadStrategy: normal, platformName: windows,
   proxy: Proxy(), se:cdp: ws://localhost:49771/devtoo..., se:cdpVersion:
   151.0.4129.72, setWindowRect: true, strictFileInteractability: false, timeouts: {
   implicit: 0, pageLoad: 300000, script: 30000}, unhandledPromptBehavior: dismiss and
   notify, webauthn:extension:credBlob: true, webauthn:extension:largeBlob: true,
   webauthn:extension:minPinLength: true, webauthn:extension:prf: true, webauthn:
   virtualAuthenticators: true}
10 Session ID: 2899188fbc0ff7b8b592e7b0aa6c9f27
11 at org.openqa.selenium.support.ui.WebDriverWait.timeoutException(WebDriverWait.java
   :85)
12 at org.openqa.selenium.support.ui.FluentWait.until(FluentWait.java:234)
13 at ui.pumaTests.new_Projekt._02_NewProjektTest.sollteNeuesProjektErstellen(
   _02_NewProjektTest.java:53)
14 at java.base/java.lang.reflect.Method.invoke(Method.java:565)
15 at java.base/java.util.ArrayList.forEach(ArrayList.java:1604)
16 at java.base/java.util.ArrayList.forEach(ArrayList.java:1604)
```

Quellcode 4: Selenium – Neues Projekt (Exemplarische Fehlerausgabe)

```
1 [SELENIUM] Setup gestartet: Anmeldung wird bei Bedarf durchgeführt und implizite
   Wartezeit deaktiviert.
2 Aug. 12, 2026 10:43:11 AM org.openqa.selenium.devtools.CdpVersionFinder.findNearestMatch
3 WARNUNG: Unable to find an exact match for CDP version 151, returning the closest
   version; found: 147; Please update to a Selenium version that supports CDP version
   151
4 [SELENIUM] Login-Seite geöffnet
5 [SELENIUM] Passwort eingegeben
6 [SELENIUM] Login-Button geklickt
7 [SELENIUM] Home-Seite wurde erreicht
8 [SELENIUM] Setup abgeschlossen: Benutzer ist angemeldet und explizite Waits werden
   verwendet.
9 [SELENIUM] Projektanlage gestartet: Projekt-Dropdown wird geöffnet.
```

```

10 [SELENIUM] Navigation zur Maske 'Neues Projekt': Link im Projekt-Dropdown wird ausgewä
    hlt.
11 [SELENIUM] Prüfung der Navigation: URL muss die Projekt-Erfassungsseite 'createProject'
    enthalten.
12 [SELENIUM] Projektname wird eingetragen und direkt gegen den Feldwert geprüft.
13 [SELENIUM] Bereich wird ausgewählt: Informationstechnologie (IT).
14 [SELENIUM] Dropdown 'project_divisionId' wird geöffnet und Option mit Text '
    Informationstechnologie (IT)' ausgewählt.
15 [SELENIUM] Dropdown 'project_divisionId' wurde erfolgreich gesetzt auf:
    Informationstechnologie (IT)
16 [SELENIUM] Projektprofil wird ausgewählt: ZEW.
17 [SELENIUM] Dropdown 'project_sapProjectProfile' wird geöffnet und Option mit Text 'ZEW'
    ausgewählt.
18 [SELENIUM] Dropdown 'project_sapProjectProfile' wurde erfolgreich gesetzt auf: ZEW -
    Entwicklungsprojekt
19 [SELENIUM] Profitcenter wird ausgewählt: CC10100400 - Ausbildung.
20 [SELENIUM] Dropdown 'project_sapProfitCenter' wird geöffnet und Option mit Text '
    CC10100400 - Ausbildung' ausgewählt.
21 [SELENIUM] Dropdown 'project_sapProfitCenter' wurde erfolgreich gesetzt auf: CC10100400
    - Ausbildung
22 [SELENIUM] Enddatum wird eingetragen und gegen den Feldwert geprüft.
23 [SELENIUM] Auftraggeber wird über Select2 gesucht und ausgewählt: Marcel Licht.
24 [SELENIUM] Select2-Feld '#project_sponsor_id' wird geöffnet und nach 'Marcel Licht'
    durchsucht.
25 [SELENIUM] Select2-Feld '#project_sponsor_id' wurde erfolgreich gesetzt auf: Marcel
    Licht (Duales Studium)
26 [SELENIUM] Projektleiter wird über Select2 gesucht und ausgewählt: Marcel Licht.
27 [SELENIUM] Select2-Feld '#project_manager_id' wird geöffnet und nach 'Marcel Licht'
    durchsucht.
28 [SELENIUM] Select2-Feld '#project_manager_id' wurde erfolgreich gesetzt auf: Marcel
    Licht (Duales Studium)
29 [SELENIUM] Projekttyp wird ausgewählt: Verbesserungsprojekt.
30 [SELENIUM] Dropdown 'project_projectType' wird geöffnet und Option mit Text '
    Verbesserungsprojekt' ausgewählt.
31 [SELENIUM] Dropdown 'project_projectType' wurde erfolgreich gesetzt auf:
    Verbesserungsprojekt
32 [SELENIUM] Projektkategorie wird ausgewählt: Prozessoptimierung (Ablauforganisation).
33 [SELENIUM] Dropdown 'project_projectCategory' wird geöffnet und Option mit Text '
    Prozessoptimierung (Ablauforganisation)' ausgewählt.
34 [SELENIUM] Dropdown 'project_projectCategory' wurde erfolgreich gesetzt auf:
    Prozessoptimierung (Ablauforganisation)
35 [SELENIUM] Beschreibung wird im CKEditor beziehungsweise im passenden Fallback-Feld
    gesetzt und geprüft.
36 [SELENIUM] Rich-Text-Beschreibung wird über die CKEditor-API gesetzt, falls verfügbar.
37 [SELENIUM] Beschreibung wurde erfolgreich über die CKEditor-API gesetzt.
38 [SELENIUM] Formularzustand wird vor dem Speichern gesammelt geprüft.
39 [SELENIUM] Projekt wird gespeichert: Speichern-Button wird gesucht, sichtbar
    positioniert und geklickt.
40 [SELENIUM] Speicher-Erfolg wird geprüft: Anwendung darf nicht mehr auf der
    Erfassungsseite stehen.
41 [SELENIUM] Speicher-Erfolg wird geprüft: Projektname muss auf der Folgeseite angezeigt
    werden.
42 [SELENIUM] Projektanlage erfolgreich abgeschlossen.

```

Quellcode 5: Selenium – Neues Projekt (Erfolgreich)

```

1 [SELENIUM] Basis-Setup gestartet: Login wird bei Bedarf durchgeführt und implizite
    Selenium-Wartezeit deaktiviert.
2 Aug. 12, 2026 10:44:47 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
3 WARNUNG: Unable to find an exact match for CDP version 151, returning the closest
    version; found: 147; Please update to a Selenium version that supports CDP version
    151
4 [SELENIUM] Login-Seite geöffnet
5 [SELENIUM] Passwort eingegeben
6 [SELENIUM] Login-Button geklickt
7 [SELENIUM] Home-Seite wurde erreicht
8 [SELENIUM] Basis-Setup abgeschlossen: Test verwendet explizite Waits für stabile UI-
    Interaktionen.
9 [SELENIUM] Activity-Anlage gestartet: Seite zum Anlegen einer neuen Activity im
    Projektplan wird geöffnet.
10 [SELENIUM] Neue Activity anlegen gestartet: Projekt wird gesucht. Projektname: Selenium
    Testprojekt

```

```

11 [SELENIUM] Projektsuche gestartet: Projekt-Suchseite wird geöffnet. Projektname:
    Selenium Testprojekt
12 [SELENIUM] Projekt-Suchseite wird geöffnet: Startseite öffnen, Projekt-Dropdown öffnen
    und Menüpunkt 'Suche' auswählen.
13 [SELENIUM] Startseite wird geöffnet und anschließend über die URL geprüft.
14 [SELENIUM] Base-URL wird ermittelt: zuerst System-Property 'baseUrl', danach
    Umgebungsvariable 'BASE_URL', sonst Default-URL.
15 [SELENIUM] Base-URL wurde ermittelt: http://localhost:8082/puma
16 [SELENIUM] Startseite wurde erfolgreich geöffnet: http://localhost:8082/puma/home
17 [SELENIUM] Projekt-Dropdown wird geöffnet: Toggle, Menü und geöffneter Zustand werden
    geprüft.
18 [SELENIUM] Projekt-Dropdown-Toggle wird gesucht und auf Klickbarkeit geprüft.
19 [SELENIUM] Projekt-Dropdown-Toggle wird robust geklickt.
20 [SELENIUM] Robuster Klick gestartet: Element wird gesucht und bei Bedarf per JavaScript
    angeklickt. Locator: By.id: project-dropdown-toggle
21 [SELENIUM] Element wurde gefunden und wird in den sichtbaren Bereich gescrollt. Locator:
    By.id: project-dropdown-toggle
22 [SELENIUM] Element wird mit normalem Selenium-Klick angeklickt. Locator: By.id: project-
    dropdown-toggle
23 [SELENIUM] Robuster Klick wurde erfolgreich ausgeführt. Locator: By.id: project-dropdown
    -toggle
24 [SELENIUM] Warten, bis das Projekt-Dropdown geöffnet ist und das Menü sichtbar angezeigt
    wird.
25 [SELENIUM] Projekt-Dropdown-Menü wird nach dem Öffnen auf Sichtbarkeit geprüft.
26 [SELENIUM] Geöffneter Zustand des Projekt-Dropdowns wird anhand von CSS-Klasse und aria-
    expanded validiert.
27 [SELENIUM] Projekt-Dropdown wurde erfolgreich geöffnet und validiert.
28 [SELENIUM] Menüpunkt 'Suche' im Projekt-Dropdown wird gesucht.
29 [SELENIUM] Menüpunkt 'Suche' wird geklickt.
30 [SELENIUM] Robuster Klick gestartet: Element wird gesucht und bei Bedarf per JavaScript
    angeklickt. Locator: By.xpath: //li[@id='project-dropdown']/ul[@id='project-
    dropdown-menu']/a[@href='projectList' and normalize-space()='Suche']
31 [SELENIUM] Element wurde gefunden und wird in den sichtbaren Bereich gescrollt. Locator:
    By.xpath: //li[@id='project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='
    projectList' and normalize-space()='Suche']
32 [SELENIUM] Element wird mit normalem Selenium-Klick angeklickt. Locator: By.xpath: //li[
    @id='project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='projectList' and
    normalize-space()='Suche']
33 [SELENIUM] Robuster Klick wurde erfolgreich ausgeführt. Locator: By.xpath: //li[@id='
    project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='projectList' and
    normalize-space()='Suche']
34 [SELENIUM] Navigation zur Projektliste wird geprüft: URL muss 'projectList' enthalten.
35 [SELENIUM] Projekt-Suchseite wurde erfolgreich geöffnet.
36 [SELENIUM] Projektname wird in das Suchfeld eingetragen. Projektname: Selenium
    Testprojekt
37 [SELENIUM] Eingabefeld wird befüllt und geprüft: Projekt-Suchfeld '#query_name' mit Wert
    : Selenium Testprojekt
38 [SELENIUM] Eingabefeld wird geleert: Projekt-Suchfeld '#query_name'
39 [SELENIUM] Wert wird in das Eingabefeld geschrieben: Projekt-Suchfeld '#query_name'
40 [SELENIUM] Input-, Change- und Blur-Events werden ausgelöst, damit die Anwendung den
    Wert verarbeitet: Projekt-Suchfeld '#query_name'
41 [SELENIUM] Eingabefeld wurde erfolgreich befüllt und geprüft: Projekt-Suchfeld '#
    query_name' = Selenium Testprojekt
42 [SELENIUM] Suche wird über den Button 'Suchen' gestartet.
43 [SELENIUM] Button wird anhand des sichtbaren Textes gesucht und geklickt: Suchen
44 [SELENIUM] Button wird in den sichtbaren Bereich gescrollt: Suchen
45 [SELENIUM] Button wird mit normalem Selenium-Klick geklickt: Suchen
46 [SELENIUM] Button wurde erfolgreich geklickt: Suchen
47 [SELENIUM] Warten auf die Ergebnistabelle der Projektsuche.
48 [SELENIUM] Prüfung, ob die Anwendung nach der Suche weiterhin auf der Projektliste ist.
49 [SELENIUM] Suchfeld wird nach der Suche geprüft: Der eingegebene Projektname muss
    weiterhin enthalten sein.
50 [SELENIUM] Suchergebnis wird geprüft: Das gesuchte Projekt muss in der Ergebnistabelle
    vorhanden sein.
51 [SELENIUM] Prüfung der Projektsuchergebnisse gestartet: Erwartetes Projekt: Selenium
    Testprojekt
52 [SELENIUM] Ergebnistabelle '#searchResult' wird gesucht und auf Sichtbarkeit geprüft.
53 [SELENIUM] Tabellenkörper der Ergebnistabelle wird erwartet.
54 [SELENIUM] Passende Tabellenzeile mit dem exakten Projektnamen wird gesucht: Selenium
    Testprojekt
55 [SELENIUM] Gefundener Projektname wird gegen den erwarteten Projektnamen geprüft.
56 [SELENIUM] Projekt wurde in der Ergebnistabelle gefunden und erfolgreich geprüft:
    Selenium Testprojekt

```

57 [SELENIUM] Projektsuche erfolgreich abgeschlossen. Projektname: Selenium Testprojekt
 58 [SELENIUM] Projekt wird aus dem Suchergebnis geöffnet und die Projektseite wird geprüft.
 Projektname: Selenium Testprojekt
 59 [SELENIUM] Projekt wird aus dem Suchergebnis geöffnet: Ergebniszeile und Öffnen-Link
 werden gesucht. Projektname: Selenium Testprojekt
 60 [SELENIUM] Ergebniszeile für das gesuchte Projekt wird gesucht.
 61 [SELENIUM] Öffnen-Link in der Ergebniszeile wird gesucht.
 62 [SELENIUM] Öffnen-Link wird in den sichtbaren Bereich gescrollt.
 63 [SELENIUM] Projekt wird über den Öffnen-Link geöffnet. Falls der normale Klick fehlschlä
 gt, wird JavaScript verwendet.
 64 [SELENIUM] Warten, bis die Projektansicht geöffnet ist und die Suchseite verlassen wurde
 .
 65 [SELENIUM] Sichtbarkeit eines Elements wird geprüft. Locator: By.id: searchResult
 66 [SELENIUM] Sichtbarkeitsprüfung abgeschlossen. Locator: By.id: searchResult, sichtbar:
 false
 67 [SELENIUM] Sichtbarkeit eines Elements wird geprüft. Locator: By.id: query_name
 68 [SELENIUM] Sichtbarkeitsprüfung abgeschlossen. Locator: By.id: query_name, sichtbar:
 false
 69 [SELENIUM] Projekt wurde erfolgreich aus dem Suchergebnis geöffnet: Selenium Testprojekt
 70 [SELENIUM] Link zum Anlegen einer neuen Activity wird gesucht.
 71 [SELENIUM] Link zum Anlegen einer neuen Activity wird in den sichtbaren Bereich
 gescrollt.
 72 [SELENIUM] Link zum Anlegen einer neuen Activity wird per JavaScript geklickt.
 73 [SELENIUM] Warten auf Activity-Editor und Prüfung, ob das Titelfeld vorhanden ist.
 74 [SELENIUM] Activity-Editor wurde erfolgreich geöffnet.
 75 [SELENIUM] Activity-Formular wird geladen und auf Verfügbarkeit geprüft.
 76 [SELENIUM] Warten auf Activity-Formular: Titelfeld muss sichtbar sein.
 77 [SELENIUM] Activity-Formular ist sichtbar.
 78 [SELENIUM] Aktivität-Bezeichnung wird eingetragen: Test Projekt Plan.
 79 [SELENIUM] Textfeld wird befüllt: Feld 'Aktivität Bezeichnung' mit Wert: Test Projekt
 Plan
 80 [SELENIUM] Eingabefeld wird befüllt und geprüft: Feld 'Aktivität Bezeichnung' mit Wert:
 Test Projekt Plan
 81 [SELENIUM] Eingabefeld wird geleert: Feld 'Aktivität Bezeichnung'
 82 [SELENIUM] Wert wird in das Eingabefeld geschrieben: Feld 'Aktivität Bezeichnung'
 83 [SELENIUM] Input-, Change- und Blur-Events werden ausgelöst, damit die Anwendung den
 Wert verarbeitet: Feld 'Aktivität Bezeichnung'
 84 [SELENIUM] Eingabefeld wurde erfolgreich befüllt und geprüft: Feld 'Aktivität
 Bezeichnung' = Test Projekt Plan
 85 [SELENIUM] Textfeld wurde erfolgreich befüllt: Feld 'Aktivität Bezeichnung'
 86 [SELENIUM] Activity-Beschreibung wird im CKEditor eingetragen.
 87 [SELENIUM] CKEditor-Beschreibung wird befüllt: sichtbares Editor-iframe wird gesucht.
 88 [SELENIUM] In das sichtbare CKEditor-iframe wird gewechselt.
 89 [SELENIUM] Body des CKEditors wird gesucht und fokussiert.
 90 [SELENIUM] CKEditor-Inhalt wird über vorhandenen Absatz oder Body aktiviert.
 91 [SELENIUM] Beschreibungstext wird in den CKEditor geschrieben.
 92 [SELENIUM] CKEditor-Beschreibung wurde erfolgreich geschrieben und geprüft.
 93 [SELENIUM] Zurück in den Hauptinhalt der Seite wechseln.
 94 [SELENIUM] Startdatum der Activity wird gesetzt: 30.06.2026.
 95 [SELENIUM] Textfeld wird befüllt: Feld 'Startdatum' mit Wert: 30.06.2026
 96 [SELENIUM] Eingabefeld wird befüllt und geprüft: Feld 'Startdatum' mit Wert: 30.06.2026
 97 [SELENIUM] Eingabefeld wird geleert: Feld 'Startdatum'
 98 [SELENIUM] Wert wird in das Eingabefeld geschrieben: Feld 'Startdatum'
 99 [SELENIUM] Input-, Change- und Blur-Events werden ausgelöst, damit die Anwendung den
 Wert verarbeitet: Feld 'Startdatum'
 100 [SELENIUM] Eingabefeld wurde erfolgreich befüllt und geprüft: Feld 'Startdatum' =
 30.06.2026
 101 [SELENIUM] Textfeld wurde erfolgreich befüllt: Feld 'Startdatum'
 102 [SELENIUM] Enddatum der Activity wird gesetzt: 16.08.2026.
 103 [SELENIUM] Textfeld wird befüllt: Feld 'Enddatum' mit Wert: 16.08.2026
 104 [SELENIUM] Eingabefeld wird befüllt und geprüft: Feld 'Enddatum' mit Wert: 16.08.2026
 105 [SELENIUM] Eingabefeld wird geleert: Feld 'Enddatum'
 106 [SELENIUM] Wert wird in das Eingabefeld geschrieben: Feld 'Enddatum'
 107 [SELENIUM] Input-, Change- und Blur-Events werden ausgelöst, damit die Anwendung den
 Wert verarbeitet: Feld 'Enddatum'
 108 [SELENIUM] Eingabefeld wurde erfolgreich befüllt und geprüft: Feld 'Enddatum' =
 16.08.2026
 109 [SELENIUM] Textfeld wurde erfolgreich befüllt: Feld 'Enddatum'
 110 [SELENIUM] Geplante Stunden werden eingetragen: 14.
 111 [SELENIUM] Zahlenfeld wird befüllt: Feld 'Geplante Stunden' mit Wert: 14
 112 [SELENIUM] Eingabefeld wird befüllt und geprüft: Feld 'Geplante Stunden' mit Wert: 14
 113 [SELENIUM] Eingabefeld wird geleert: Feld 'Geplante Stunden'
 114 [SELENIUM] Wert wird in das Eingabefeld geschrieben: Feld 'Geplante Stunden'

```

115 [SELENIUM] Input-, Change- und Blur-Events werden ausgelöst, damit die Anwendung den
    Wert verarbeitet: Feld 'Geplante Stunden'
116 [SELENIUM] Eingabefeld wurde erfolgreich befüllt und geprüft: Feld 'Geplante Stunden' =
    14
117 [SELENIUM] Zahlenfeld wurde erfolgreich befüllt: Feld 'Geplante Stunden'
118 [SELENIUM] Arbeitsplatz wird ausgewählt: PS06110 - Globale IT-Services Administration.
119 [SELENIUM] Dropdown wird gesucht und sichtbarer Text wird ausgewählt: Feld 'Arbeitsplatz'
    = PS06110 - Globale IT-Services Administration
120 [SELENIUM] Dropdown wurde erfolgreich ausgewählt und geprüft: Feld 'Arbeitsplatz' =
    PS06110 - Globale IT-Services Administration
121 [SELENIUM] Wirkliches Startdatum wird gesetzt: 15.06.2026.
122 [SELENIUM] Textfeld wird befüllt: Feld 'Wirkliches Startdatum' mit Wert: 15.06.2026
123 [SELENIUM] Eingabefeld wird befüllt und geprüft: Feld 'Wirkliches Startdatum' mit Wert:
    15.06.2026
124 [SELENIUM] Eingabefeld wird geleert: Feld 'Wirkliches Startdatum'
125 [SELENIUM] Wert wird in das Eingabefeld geschrieben: Feld 'Wirkliches Startdatum'
126 [SELENIUM] Input-, Change- und Blur-Events werden ausgelöst, damit die Anwendung den
    Wert verarbeitet: Feld 'Wirkliches Startdatum'
127 [SELENIUM] Eingabefeld wurde erfolgreich befüllt und geprüft: Feld 'Wirkliches
    Startdatum' = 15.06.2026
128 [SELENIUM] Textfeld wurde erfolgreich befüllt: Feld 'Wirkliches Startdatum'
129 [SELENIUM] Wirkliches Enddatum wird gesetzt: 17.08.2026.
130 [SELENIUM] Textfeld wird befüllt: Feld 'Wirkliches Enddatum' mit Wert: 17.08.2026
131 [SELENIUM] Eingabefeld wird befüllt und geprüft: Feld 'Wirkliches Enddatum' mit Wert:
    17.08.2026
132 [SELENIUM] Eingabefeld wird geleert: Feld 'Wirkliches Enddatum'
133 [SELENIUM] Wert wird in das Eingabefeld geschrieben: Feld 'Wirkliches Enddatum'
134 [SELENIUM] Input-, Change- und Blur-Events werden ausgelöst, damit die Anwendung den
    Wert verarbeitet: Feld 'Wirkliches Enddatum'
135 [SELENIUM] Eingabefeld wurde erfolgreich befüllt und geprüft: Feld 'Wirkliches Enddatum'
    = 17.08.2026
136 [SELENIUM] Textfeld wurde erfolgreich befüllt: Feld 'Wirkliches Enddatum'
137 [SELENIUM] Verbliebene Stunden werden eingetragen: 13.
138 [SELENIUM] Zahlenfeld wird befüllt: Feld 'Verbliebene Stunden' mit Wert: 13
139 [SELENIUM] Eingabefeld wird befüllt und geprüft: Feld 'Verbliebene Stunden' mit Wert: 13
140 [SELENIUM] Eingabefeld wird geleert: Feld 'Verbliebene Stunden'
141 [SELENIUM] Wert wird in das Eingabefeld geschrieben: Feld 'Verbliebene Stunden'
142 [SELENIUM] Input-, Change- und Blur-Events werden ausgelöst, damit die Anwendung den
    Wert verarbeitet: Feld 'Verbliebene Stunden'
143 [SELENIUM] Eingabefeld wurde erfolgreich befüllt und geprüft: Feld 'Verbliebene Stunden'
    = 13
144 [SELENIUM] Zahlenfeld wurde erfolgreich befüllt: Feld 'Verbliebene Stunden'
145 [SELENIUM] Fortschritt der Activity wird eingetragen: 4 Prozent.
146 [SELENIUM] Zahlenfeld wird befüllt: Feld 'Fortschritt in %' mit Wert: 4
147 [SELENIUM] Eingabefeld wird befüllt und geprüft: Feld 'Fortschritt in %' mit Wert: 4
148 [SELENIUM] Eingabefeld wird geleert: Feld 'Fortschritt in %'
149 [SELENIUM] Wert wird in das Eingabefeld geschrieben: Feld 'Fortschritt in %'
150 [SELENIUM] Input-, Change- und Blur-Events werden ausgelöst, damit die Anwendung den
    Wert verarbeitet: Feld 'Fortschritt in %'
151 [SELENIUM] Eingabefeld wurde erfolgreich befüllt und geprüft: Feld 'Fortschritt in %' =
    4
152 [SELENIUM] Zahlenfeld wurde erfolgreich befüllt: Feld 'Fortschritt in %'
153 [SELENIUM] Offene Kosten werden eingetragen: 0.
154 [SELENIUM] Zahlenfeld wird befüllt: Feld 'Offene Kosten' mit Wert: 0
155 [SELENIUM] Eingabefeld wird befüllt und geprüft: Feld 'Offene Kosten' mit Wert: 0
156 [SELENIUM] Eingabefeld wird geleert: Feld 'Offene Kosten'
157 [SELENIUM] Wert wird in das Eingabefeld geschrieben: Feld 'Offene Kosten'
158 [SELENIUM] Input-, Change- und Blur-Events werden ausgelöst, damit die Anwendung den
    Wert verarbeitet: Feld 'Offene Kosten'
159 [SELENIUM] Eingabefeld wurde erfolgreich befüllt und geprüft: Feld 'Offene Kosten' = 0
160 [SELENIUM] Zahlenfeld wurde erfolgreich befüllt: Feld 'Offene Kosten'
161 [SELENIUM] Neue Activity wird gespeichert und Abschluss der Speicherung wird abgewartet.
162 [SELENIUM] Speichern der Activity gestartet: Speichern-Button wird gesucht.
163 [SELENIUM] Speichern-Button wird in den sichtbaren Bereich gescrollt.
164 [SELENIUM] Speichern-Button wird per JavaScript geklickt.
165 [SELENIUM] Nach dem Speichern wird gewartet, bis das Formular verschwindet oder die
    Seite sich ändert.
166 [SELENIUM] Speichern der Activity abgeschlossen: Folgezustand wurde erkannt.
167 [SELENIUM] Activity-Anlage erfolgreich abgeschlossen.

```

Quellcode 6: Selenium – Neue Activity (Erfolgreich)

```
1 _03_EditActivityTest (1)
```

```

2 ui.pumaTests._edit_Data._03_EditActivityTest
3 editActivity(ui.pumaTests._edit_Data._03_EditActivityTest)
4 org.openqa.selenium.TimeoutException: Expected condition failed: waiting for presence of
   element found by By.cssSelector: a.openEntry
5 (tried for 10 seconds with 500 milliseconds interval)
6 Build info: version: '4.43.0', revision: 'dd0f534'
7 System info: os.name: 'Windows 11', os.arch: 'amd64', os.version: '10.0', java.version:
   '25.0.2'
8 Driver info: org.openqa.selenium.edge.EdgeDriver
9 Capabilities {acceptInsecureCerts: false, browserName: MicrosoftEdge, browserVersion:
   151.0.4129.72, fedcm:accounts: true, goog:processID: 27668, ms:edgeOptions: {
   debuggerAddress: localhost:55075}, msedge: {msedgedriverVersion: 151.0.4129.78 (
   dd5b3842e0c9..., userDataDir: C:\Users\MARCEL~1.LIC\AppData...},
   networkConnectionEnabled: false, pageLoadStrategy: normal, platformName: windows,
   proxy: Proxy(), se:cdp: ws://localhost:55075/devtoo..., se:cdpVersion:
   151.0.4129.72, setWindowRect: true, strictFileInteractability: false, timeouts: {
   implicit: 0, pageLoad: 300000, script: 30000}, unhandledPromptBehavior: dismiss and
   notify, webauthn:extension:credBlob: true, webauthn:extension:largeBlob: true,
   webauthn:extension:minPinLength: true, webauthn:extension:prf: true, webauthn:
   virtualAuthenticators: true}
10 Session ID: d478c127beb97af33d87aaff87c0482b
11 at org.openqa.selenium.support.ui.WebDriverWait.timeoutException(WebDriverWait.java
   :85)
12 at org.openqa.selenium.support.ui.FluentWait.until(FluentWait.java:234)
13 at ui.pumaTests.common.BaseEditActivity.openActivityFromProject(BaseEditActivity.java
   :32)
14 at ui.pumaTests._edit_Data._03_EditActivityTest.editActivity(_03_EditActivityTest.java
   :16)
15 at java.base/java.lang.reflect.Method.invoke(Method.java:565)
16 at java.base/java.util.ArrayList.forEach(ArrayList.java:1604)
17 at java.base/java.util.ArrayList.forEach(ArrayList.java:1604)

```

Quellcode 7: Selenium – Activity bearbeiten (Exemplarische Fehlerausgabe)

```

1 [SELENIUM] Basis-Setup gestartet: Login wird bei Bedarf durchgeführt und implizite
   Selenium-Wartezeit deaktiviert.
2 Aug. 12, 2026 10:59:05 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
3 WARNUNG: Unable to find an exact match for CDP version 151, returning the closest
   version; found: 147; Please update to a Selenium version that supports CDP version
   151
4 [SELENIUM] Login-Seite geöffnet
5 [SELENIUM] Passwort eingegeben
6 [SELENIUM] Login-Button geklickt
7 [SELENIUM] Home-Seite wurde erreicht
8 [SELENIUM] Basis-Setup abgeschlossen: Test verwendet explizite Waits für stabile UI-
   Interaktionen.
9 [SELENIUM] Activity-Bearbeitung gestartet: Activity wird aus dem Projekt geöffnet.
10 [SELENIUM] Activity öffnen gestartet: Das Projekt wird gesucht, geöffnet und anschließ
   end die erste vorhandene Activity geöffnet.
11 [SELENIUM] Projekt wird gesucht: Selenium Testprojekt
12 [SELENIUM] Projektsuche gestartet: Projekt-Suchseite wird geöffnet. Projektname:
   Selenium Testprojekt
13 [SELENIUM] Projekt-Suchseite wird geöffnet: Startseite öffnen, Projekt-Dropdown öffnen
   und Menüpunkt 'Suche' auswählen.
14 [SELENIUM] Startseite wird geöffnet und anschließend über die URL geprüft.
15 [SELENIUM] Base-URL wird ermittelt: zuerst System-Property 'baseUrl', danach
   Umgebungsvariable 'BASE_URL', sonst Default-URL.
16 [SELENIUM] Base-URL wurde ermittelt: http://localhost:8082/puma
17 [SELENIUM] Startseite wurde erfolgreich geöffnet: http://localhost:8082/puma/home
18 [SELENIUM] Projekt-Dropdown wird geöffnet: Toggle, Menü und öffneter Zustand werden
   geprüft.
19 [SELENIUM] Projekt-Dropdown-Toggle wird gesucht und auf Klickbarkeit geprüft.
20 [SELENIUM] Projekt-Dropdown-Toggle wird robust geklickt.
21 [SELENIUM] Robuster Klick gestartet: Element wird gesucht und bei Bedarf per JavaScript
   angeklickt. Locator: By.id: project-dropdown-toggle
22 [SELENIUM] Element wurde gefunden und wird in den sichtbaren Bereich gescrollt. Locator:
   By.id: project-dropdown-toggle
23 [SELENIUM] Element wird mit normalem Selenium-Klick angeklickt. Locator: By.id: project-
   dropdown-toggle
24 [SELENIUM] Robuster Klick wurde erfolgreich ausgeführt. Locator: By.id: project-dropdown
   -toggle
25 [SELENIUM] Warten, bis das Projekt-Dropdown geöffnet ist und das Menü sichtbar angezeigt
   wird.

```

26 [SELENIUM] Projekt-Dropdown-Menü wird nach dem Öffnen auf Sichtbarkeit geprüft.
27 [SELENIUM] Geöffneter Zustand des Projekt-Dropdowns wird anhand von CSS-Klasse und aria-expanded validiert.
28 [SELENIUM] Projekt-Dropdown wurde erfolgreich geöffnet und validiert.
29 [SELENIUM] Menüpunkt 'Suche' im Projekt-Dropdown wird gesucht.
30 [SELENIUM] Menüpunkt 'Suche' wird geklickt.
31 [SELENIUM] Robuster Klick gestartet: Element wird gesucht und bei Bedarf per JavaScript angeklickt. Locator: By.xpath: //li[@id='project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='projectList' and normalize-space()='Suche']
32 [SELENIUM] Element wurde gefunden und wird in den sichtbaren Bereich gescrollt. Locator: By.xpath: //li[@id='project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='projectList' and normalize-space()='Suche']
33 [SELENIUM] Element wird mit normalem Selenium-Klick angeklickt. Locator: By.xpath: //li[@id='project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='projectList' and normalize-space()='Suche']
34 [SELENIUM] Robuster Klick wurde erfolgreich ausgeführt. Locator: By.xpath: //li[@id='project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='projectList' and normalize-space()='Suche']
35 [SELENIUM] Navigation zur Projektliste wird geprüft: URL muss 'projectList' enthalten.
36 [SELENIUM] Projekt-Suchseite wurde erfolgreich geöffnet.
37 [SELENIUM] Projektname wird in das Suchfeld eingetragen. Projektname: Selenium Testprojekt
38 [SELENIUM] Eingabefeld wird befüllt und geprüft: Projekt-Suchfeld '#query_name' mit Wert : Selenium Testprojekt
39 [SELENIUM] Eingabefeld wird geleert: Projekt-Suchfeld '#query_name'
40 [SELENIUM] Wert wird in das Eingabefeld geschrieben: Projekt-Suchfeld '#query_name'
41 [SELENIUM] Input-, Change- und Blur-Events werden ausgelöst, damit die Anwendung den Wert verarbeitet: Projekt-Suchfeld '#query_name'
42 [SELENIUM] Eingabefeld wurde erfolgreich befüllt und geprüft: Projekt-Suchfeld '#query_name' = Selenium Testprojekt
43 [SELENIUM] Suche wird über den Button 'Suchen' gestartet.
44 [SELENIUM] Button wird anhand des sichtbaren Textes gesucht und geklickt: Suchen
45 [SELENIUM] Button wird in den sichtbaren Bereich gescrollt: Suchen
46 [SELENIUM] Button wird mit normalem Selenium-Klick geklickt: Suchen
47 [SELENIUM] Button wurde erfolgreich geklickt: Suchen
48 [SELENIUM] Warten auf die Ergebnistabelle der Projektsuche.
49 [SELENIUM] Prüfung, ob die Anwendung nach der Suche weiterhin auf der Projektliste ist.
50 [SELENIUM] Suchfeld wird nach der Suche geprüft: Der eingegebene Projektname muss weiterhin enthalten sein.
51 [SELENIUM] Suchergebnis wird geprüft: Das gesuchte Projekt muss in der Ergebnistabelle vorhanden sein.
52 [SELENIUM] Prüfung der Projektsuchergebnisse gestartet: Erwartetes Projekt: Selenium Testprojekt
53 [SELENIUM] Ergebnistabelle '#searchResult' wird gesucht und auf Sichtbarkeit geprüft.
54 [SELENIUM] Tabellenkörper der Ergebnistabelle wird erwartet.
55 [SELENIUM] Passende Tabellenzeile mit dem exakten Projektnamen wird gesucht: Selenium Testprojekt
56 [SELENIUM] Gefundener Projektname wird gegen den erwarteten Projektnamen geprüft.
57 [SELENIUM] Projekt wurde in der Ergebnistabelle gefunden und erfolgreich geprüft: Selenium Testprojekt
58 [SELENIUM] Projektsuche erfolgreich abgeschlossen. Projektname: Selenium Testprojekt
59 [SELENIUM] Projekt wird aus dem Suchergebnis geöffnet und auf korrekte Anzeige geprüft: Selenium Testprojekt
60 [SELENIUM] Projekt wird aus dem Suchergebnis geöffnet: Ergebniszeile und Öffnen-Link werden gesucht. Projektname: Selenium Testprojekt
61 [SELENIUM] Ergebniszeile für das gesuchte Projekt wird gesucht.
62 [SELENIUM] Öffnen-Link in der Ergebniszeile wird gesucht.
63 [SELENIUM] Öffnen-Link wird in den sichtbaren Bereich gescrollt.
64 [SELENIUM] Projekt wird über den Öffnen-Link geöffnet. Falls der normale Klick fehlschlägt, wird JavaScript verwendet.
65 [SELENIUM] Warten, bis die Projektansicht geöffnet ist und die Suchseite verlassen wurde.
66 [SELENIUM] Sichtbarkeit eines Elements wird geprüft. Locator: By.id: searchResult
67 [SELENIUM] Sichtbarkeitsprüfung abgeschlossen. Locator: By.id: searchResult, sichtbar: false
68 [SELENIUM] Sichtbarkeit eines Elements wird geprüft. Locator: By.id: query_name
69 [SELENIUM] Sichtbarkeitsprüfung abgeschlossen. Locator: By.id: query_name, sichtbar: false
70 [SELENIUM] Projekt wurde erfolgreich aus dem Suchergebnis geöffnet: Selenium Testprojekt
71 [SELENIUM] Link zur Activity-Übersicht wird gesucht.
72 [SELENIUM] Activity-Übersicht wird geöffnet.
73 [SELENIUM] Warten auf vorhandene Activity-Einträge in der Übersicht.
74 [SELENIUM] Erste vorhandene Activity wird aus der Übersicht geöffnet.


```

75 [SELENIUM] Activity wurde erfolgreich aus dem Projekt geöffnet.
76 [SELENIUM] Bearbeitungsmodus wird geöffnet.
77 [SELENIUM] Bearbeitungsmodus wird gestartet: Bearbeiten-Button wird gesucht, sichtbar
    positioniert und geklickt.
78 [SELENIUM] Bearbeiten-Button wird in den sichtbaren Bereich gescrollt.
79 [SELENIUM] Bearbeiten-Button wird per JavaScript geklickt.
80 [SELENIUM] Bearbeitungsmodus wurde erfolgreich geöffnet.
81 [SELENIUM] Fortschritt wird auf 15 Prozent gesetzt.
82 [SELENIUM] Zahlenfeld wird befüllt. Locator: By.id: activity.progress, Wert: 15
83 [SELENIUM] Feld wird befüllt: Element wird gesucht. Locator: By.id: activity.progress,
    Wert: 15
84 [SELENIUM] Feld wird geleert. Locator: By.id: activity.progress
85 [SELENIUM] Wert wird in das Feld geschrieben. Locator: By.id: activity.progress, Wert:
    15
86 [SELENIUM] Feld wurde erfolgreich befüllt. Locator: By.id: activity.progress
87 [SELENIUM] Zahlenfeld wurde erfolgreich befüllt. Locator: By.id: activity.progress, Wert
    : 15
88 [SELENIUM] Tatsächliches Enddatum wird auf den 17.08.2026 gesetzt.
89 [SELENIUM] Feld wird befüllt: Element wird gesucht. Locator: By.id: activity.
    actualEndDate, Wert: 17.08.2026
90 [SELENIUM] Feld wird geleert. Locator: By.id: activity.actualEndDate
91 [SELENIUM] Wert wird in das Feld geschrieben. Locator: By.id: activity.actualEndDate,
    Wert: 17.08.2026
92 [SELENIUM] Feld wurde erfolgreich befüllt. Locator: By.id: activity.actualEndDate
93 [SELENIUM] Verbleibende Kosten werden auf 50 gesetzt.
94 [SELENIUM] Zahlenfeld wird befüllt. Locator: By.id: activityRemainingCosts, Wert: 50
95 [SELENIUM] Feld wird befüllt: Element wird gesucht. Locator: By.id:
    activityRemainingCosts, Wert: 50
96 [SELENIUM] Feld wird geleert. Locator: By.id: activityRemainingCosts
97 [SELENIUM] Wert wird in das Feld geschrieben. Locator: By.id: activityRemainingCosts,
    Wert: 50
98 [SELENIUM] Feld wurde erfolgreich befüllt. Locator: By.id: activityRemainingCosts
99 [SELENIUM] Zahlenfeld wurde erfolgreich befüllt. Locator: By.id: activityRemainingCosts,
    Wert: 50
100 [SELENIUM] Arbeitsplatz wird ausgewählt: PS02080.
101 [SELENIUM] Auswahlliste wird geöffnet: Option mit enthaltenem Text wird gesucht. Locator
    : By.id: activity_workplace, Suchtext: PS02080
102 [SELENIUM] Passende Option wurde gefunden und wird ausgewählt: PS02080 - Software II
103 [SELENIUM] Option wurde erfolgreich ausgewählt: PS02080 - Software II
104 [SELENIUM] Erster Checklistenpunkt wird ergänzt: Alle Testfälle für Cypress
    implementieren.
105 [SELENIUM] Checklistenpunkt wird hinzugefügt: Button wird gesucht. Index: 0, Text: Alle
    Testfälle für Cypress implementieren
106 [SELENIUM] Button zum Hinzufügen eines Checklistenpunktes wird per JavaScript geklickt.
107 [SELENIUM] Warten, bis der neue Checklistenpunkt in der Tabelle vorhanden ist. Index: 0
108 [SELENIUM] Text des Checklistenpunktes wird eingetragen. Index: 0, Text: Alle Testfälle
    für Cypress implementieren
109 [SELENIUM] Checklistenpunkt wurde erfolgreich hinzugefügt. Index: 0, Text: Alle Testfä
    lle für Cypress implementieren
110 [SELENIUM] Zweiter Checklistenpunkt wird ergänzt: Mit schriftlicher Ausarbeitung der
    Fallstudie beginnen.
111 [SELENIUM] Checklistenpunkt wird hinzugefügt: Button wird gesucht. Index: 1, Text: Mit
    schriftlicher Ausarbeitung der Fallstudie beginnen
112 [SELENIUM] Button zum Hinzufügen eines Checklistenpunktes wird per JavaScript geklickt.
113 [SELENIUM] Warten, bis der neue Checklistenpunkt in der Tabelle vorhanden ist. Index: 1
114 [SELENIUM] Text des Checklistenpunktes wird eingetragen. Index: 1, Text: Mit
    schriftlicher Ausarbeitung der Fallstudie beginnen
115 [SELENIUM] Checklistenpunkt wurde erfolgreich hinzugefügt. Index: 1, Text: Mit
    schriftlicher Ausarbeitung der Fallstudie beginnen
116 [SELENIUM] Geänderte Activity wird gespeichert.
117 [SELENIUM] Activity speichern gestartet: Speichern-Button wird gesucht, sichtbar
    positioniert und geklickt.
118 [SELENIUM] Speichern-Button wird in den sichtbaren Bereich gescrollt.
119 [SELENIUM] Speichern-Button wird per JavaScript geklickt.
120 [SELENIUM] Activity wurde gespeichert.
121 [SELENIUM] Activity-Bearbeitung erfolgreich abgeschlossen.

```

Quellcode 8: Selenium – Activity bearbeiten (Erfolgreich)

```

1 [SELENIUM] Login-Seite geöffnet
2 [SELENIUM] Passwort eingegeben
3 [SELENIUM] Login-Button geklickt
4 [SELENIUM] Home-Seite wurde erreicht

```

5 [SELENIUM] Basis-Setup abgeschlossen: Test verwendet explizite Waits für stabile UI-Interaktionen.

6 [SELENIUM] Validierung der bearbeiteten Activity gestartet: Activity wird aus dem Projekt geöffnet.

7 [SELENIUM] Activity öffnen gestartet: Das Projekt wird gesucht, geöffnet und anschließend die erste vorhandene Activity geöffnet.

8 [SELENIUM] Projekt wird gesucht: Selenium Testprojekt

9 [SELENIUM] Projektsuche gestartet: Projekt-Suchseite wird geöffnet. Projektname: Selenium Testprojekt

10 [SELENIUM] Projekt-Suchseite wird geöffnet: Startseite öffnen, Projekt-Dropdown öffnen und Menüpunkt 'Suche' auswählen.

11 [SELENIUM] Startseite wird geöffnet und anschließend über die URL geprüft.

12 [SELENIUM] Base-URL wird ermittelt: zuerst System-Property 'baseUrl', danach Umgebungsvariable 'BASE_URL', sonst Default-URL.

13 [SELENIUM] Base-URL wurde ermittelt: http://localhost:8082/puma

14 [SELENIUM] Startseite wurde erfolgreich geöffnet: http://localhost:8082/puma/home

15 [SELENIUM] Projekt-Dropdown wird geöffnet: Toggle, Menü und geöffneter Zustand werden geprüft.

16 [SELENIUM] Projekt-Dropdown-Toggle wird gesucht und auf Klickbarkeit geprüft.

17 [SELENIUM] Projekt-Dropdown-Toggle wird robust geklickt.

18 [SELENIUM] Robuster Klick gestartet: Element wird gesucht und bei Bedarf per JavaScript angeklickt. Locator: By.id: project-dropdown-toggle

19 [SELENIUM] Element wurde gefunden und wird in den sichtbaren Bereich gescrollt. Locator: By.id: project-dropdown-toggle

20 [SELENIUM] Element wird mit normalem Selenium-Klick angeklickt. Locator: By.id: project-dropdown-toggle

21 [SELENIUM] Robuster Klick wurde erfolgreich ausgeführt. Locator: By.id: project-dropdown-toggle

22 [SELENIUM] Warten, bis das Projekt-Dropdown geöffnet ist und das Menü sichtbar angezeigt wird.

23 [SELENIUM] Projekt-Dropdown-Menü wird nach dem Öffnen auf Sichtbarkeit geprüft.

24 [SELENIUM] Geöffneter Zustand des Projekt-Dropdowns wird anhand von CSS-Klasse und aria-expanded validiert.

25 [SELENIUM] Projekt-Dropdown wurde erfolgreich geöffnet und validiert.

26 [SELENIUM] Menüpunkt 'Suche' im Projekt-Dropdown wird gesucht.

27 [SELENIUM] Menüpunkt 'Suche' wird geklickt.

28 [SELENIUM] Robuster Klick gestartet: Element wird gesucht und bei Bedarf per JavaScript angeklickt. Locator: By.xpath: //li[@id='project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='projectList' and normalize-space()='Suche']

29 [SELENIUM] Element wurde gefunden und wird in den sichtbaren Bereich gescrollt. Locator: By.xpath: //li[@id='project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='projectList' and normalize-space()='Suche']

30 [SELENIUM] Element wird mit normalem Selenium-Klick angeklickt. Locator: By.xpath: //li[@id='project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='projectList' and normalize-space()='Suche']

31 [SELENIUM] Robuster Klick wurde erfolgreich ausgeführt. Locator: By.xpath: //li[@id='project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='projectList' and normalize-space()='Suche']

32 [SELENIUM] Navigation zur Projektliste wird geprüft: URL muss 'projectList' enthalten.

33 [SELENIUM] Projekt-Suchseite wurde erfolgreich geöffnet.

34 [SELENIUM] Projektname wird in das Suchfeld eingetragen. Projektname: Selenium Testprojekt

35 [SELENIUM] Eingabefeld wird befüllt und geprüft: Projekt-Suchfeld '#query_name' mit Wert : Selenium Testprojekt

36 [SELENIUM] Eingabefeld wird geleert: Projekt-Suchfeld '#query_name'

37 [SELENIUM] Wert wird in das Eingabefeld geschrieben: Projekt-Suchfeld '#query_name'

38 [SELENIUM] Input-, Change- und Blur-Events werden ausgelöst, damit die Anwendung den Wert verarbeitet: Projekt-Suchfeld '#query_name'

39 [SELENIUM] Eingabefeld wurde erfolgreich befüllt und geprüft: Projekt-Suchfeld '#query_name' = Selenium Testprojekt

40 [SELENIUM] Suche wird über den Button 'Suchen' gestartet.

41 [SELENIUM] Button wird anhand des sichtbaren Textes gesucht und geklickt: Suchen

42 [SELENIUM] Button wird in den sichtbaren Bereich gescrollt: Suchen

43 [SELENIUM] Button wird mit normalem Selenium-Klick geklickt: Suchen

44 [SELENIUM] Button wurde erfolgreich geklickt: Suchen

45 [SELENIUM] Warten auf die Ergebnistabelle der Projektsuche.

46 [SELENIUM] Prüfung, ob die Anwendung nach der Suche weiterhin auf der Projektliste ist.

47 [SELENIUM] Suchfeld wird nach der Suche geprüft: Der eingegebene Projektname muss weiterhin enthalten sein.

48 [SELENIUM] Suchergebnis wird geprüft: Das gesuchte Projekt muss in der Ergebnistabelle vorhanden sein.

49 [SELENIUM] Prüfung der Projektsuchergebnisse gestartet: Erwartetes Projekt: Selenium Testprojekt

50 [SELENIUM] Ergebnistabelle '#searchResult' wird gesucht und auf Sichtbarkeit geprüft.
51 [SELENIUM] Tabellenkörper der Ergebnistabelle wird erwartet.
52 [SELENIUM] Passende Tabellenzeile mit dem exakten Projektnamen wird gesucht: Selenium
Testprojekt
53 [SELENIUM] Gefundener Projektnamen wird gegen den erwarteten Projektnamen geprüft.
54 [SELENIUM] Projekt wurde in der Ergebnistabelle gefunden und erfolgreich geprüft:
Selenium Testprojekt
55 [SELENIUM] Projektsuche erfolgreich abgeschlossen. Projektname: Selenium Testprojekt
56 [SELENIUM] Projekt wird aus dem Suchergebnis geöffnet und auf korrekte Anzeige geprüft:
Selenium Testprojekt
57 [SELENIUM] Projekt wird aus dem Suchergebnis geöffnet: Ergebniszeile und Öffnen-Link
werden gesucht. Projektname: Selenium Testprojekt
58 [SELENIUM] Ergebniszeile für das gesuchte Projekt wird gesucht.
59 [SELENIUM] Öffnen-Link in der Ergebniszeile wird gesucht.
60 [SELENIUM] Öffnen-Link wird in den sichtbaren Bereich gescrollt.
61 [SELENIUM] Projekt wird über den Öffnen-Link geöffnet. Falls der normale Klick fehlschlä
gt, wird JavaScript verwendet.
62 [SELENIUM] Warten, bis die Projektansicht geöffnet ist und die Suchseite verlassen wurde
.
63 [SELENIUM] Sichtbarkeit eines Elements wird geprüft. Locator: By.id: searchResult
64 [SELENIUM] Sichtbarkeitsprüfung abgeschlossen. Locator: By.id: searchResult, sichtbar:
false
65 [SELENIUM] Sichtbarkeit eines Elements wird geprüft. Locator: By.id: query_name
66 [SELENIUM] Sichtbarkeitsprüfung abgeschlossen. Locator: By.id: query_name, sichtbar:
false
67 [SELENIUM] Projekt wurde erfolgreich aus dem Suchergebnis geöffnet: Selenium Testprojekt
68 [SELENIUM] Link zur Activity-Übersicht wird gesucht.
69 [SELENIUM] Activity-Übersicht wird geöffnet.
70 [SELENIUM] Warten auf vorhandene Activity-Einträge in der Übersicht.
71 [SELENIUM] Erste vorhandene Activity wird aus der Übersicht geöffnet.
72 [SELENIUM] Activity wurde erfolgreich aus dem Projekt geöffnet.
73 [SELENIUM] Fortschritt der Activity wird ausgelesen.
74 [SELENIUM] Fortschritt wird geprüft: Erwarteter Wert ist 15.
75 [SELENIUM] Validierung der bearbeiteten Activity erfolgreich abgeschlossen.
76 [SELENIUM] Cleanup gestartet: Activity-Testdaten werden zurückgesetzt.
77 [SELENIUM] Reset der Activity-Testdaten gestartet: Activity wird geöffnet, bearbeitet
und auf definierte Standardwerte zurückgesetzt.
78 [SELENIUM] Activity öffnen gestartet: Das Projekt wird gesucht, geöffnet und anschließ
end die erste vorhandene Activity geöffnet.
79 [SELENIUM] Projekt wird gesucht: Selenium Testprojekt
80 [SELENIUM] Projektsuche gestartet: Projekt-Suchseite wird geöffnet. Projektname:
Selenium Testprojekt
81 [SELENIUM] Projekt-Suchseite wird geöffnet: Startseite öffnen, Projekt-Dropdown öffnen
und Menüpunkt 'Suche' auswählen.
82 [SELENIUM] Startseite wird geöffnet und anschließend über die URL geprüft.
83 [SELENIUM] Base-URL wird ermittelt: zuerst System-Property 'baseUrl', danach
Umgebungsvariable 'BASE_URL', sonst Default-URL.
84 [SELENIUM] Base-URL wurde ermittelt: http://localhost:8082/puma
85 [SELENIUM] Startseite wurde erfolgreich geöffnet: http://localhost:8082/puma/home
86 [SELENIUM] Projekt-Dropdown wird geöffnet: Toggle, Menü und geöffneter Zustand werden
geprüft.
87 [SELENIUM] Projekt-Dropdown-Toggle wird gesucht und auf Klickbarkeit geprüft.
88 [SELENIUM] Projekt-Dropdown-Toggle wird robust geklickt.
89 [SELENIUM] Robuster Klick gestartet: Element wird gesucht und bei Bedarf per JavaScript
angeklickt. Locator: By.id: project-dropdown-toggle
90 [SELENIUM] Element wurde gefunden und wird in den sichtbaren Bereich gescrollt. Locator:
By.id: project-dropdown-toggle
91 [SELENIUM] Element wird mit normalem Selenium-Klick angeklickt. Locator: By.id: project-
dropdown-toggle
92 [SELENIUM] Robuster Klick wurde erfolgreich ausgeführt. Locator: By.id: project-dropdown
-toggle
93 [SELENIUM] Warten, bis das Projekt-Dropdown geöffnet ist und das Menü sichtbar angezeigt
wird.
94 [SELENIUM] Projekt-Dropdown-Menü wird nach dem Öffnen auf Sichtbarkeit geprüft.
95 [SELENIUM] Geöffneter Zustand des Projekt-Dropdowns wird anhand von CSS-Klasse und aria-
expanded validiert.
96 [SELENIUM] Projekt-Dropdown wurde erfolgreich geöffnet und validiert.
97 [SELENIUM] Menüpunkt 'Suche' im Projekt-Dropdown wird gesucht.
98 [SELENIUM] Menüpunkt 'Suche' wird geklickt.
99 [SELENIUM] Robuster Klick gestartet: Element wird gesucht und bei Bedarf per JavaScript
angeklickt. Locator: By.xpath: //li[@id='project-dropdown']/ul[@id='project-
dropdown-menu']/a[@href='projectList' and normalize-space()='Suche']
100 [SELENIUM] Element wurde gefunden und wird in den sichtbaren Bereich gescrollt. Locator:

```

    By.xpath: //li[@id='project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='
projectList' and normalize-space()='Suche']
101 [SELENIUM] Element wird mit normalem Selenium-Klick angeklickt. Locator: By.xpath: //li[
    @id='project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='projectList' and
    normalize-space()='Suche']
102 [SELENIUM] Robuster Klick wurde erfolgreich ausgeführt. Locator: By.xpath: //li[@id='
    project-dropdown']/ul[@id='project-dropdown-menu']/a[@href='projectList' and
    normalize-space()='Suche']
103 [SELENIUM] Navigation zur Projektliste wird geprüft: URL muss 'projectList' enthalten.
104 [SELENIUM] Projekt-Suchseite wurde erfolgreich geöffnet.
105 [SELENIUM] Projektname wird in das Suchfeld eingetragen. Projektname: Selenium
    Testprojekt
106 [SELENIUM] Eingabefeld wird befüllt und geprüft: Projekt-Suchfeld '#query_name' mit Wert
    : Selenium Testprojekt
107 [SELENIUM] Eingabefeld wird geleert: Projekt-Suchfeld '#query_name'
108 [SELENIUM] Wert wird in das Eingabefeld geschrieben: Projekt-Suchfeld '#query_name'
109 [SELENIUM] Input-, Change- und Blur-Events werden ausgelöst, damit die Anwendung den
    Wert verarbeitet: Projekt-Suchfeld '#query_name'
110 [SELENIUM] Eingabefeld wurde erfolgreich befüllt und geprüft: Projekt-Suchfeld '#
    query_name' = Selenium Testprojekt
111 [SELENIUM] Suche wird über den Button 'Suchen' gestartet.
112 [SELENIUM] Button wird anhand des sichtbaren Textes gesucht und geklickt: Suchen
113 [SELENIUM] Button wird in den sichtbaren Bereich gescrollt: Suchen
114 [SELENIUM] Button wird mit normalem Selenium-Klick geklickt: Suchen
115 [SELENIUM] Button wurde erfolgreich geklickt: Suchen
116 [SELENIUM] Warten auf die Ergebnistabelle der Projektsuche.
117 [SELENIUM] Prüfung, ob die Anwendung nach der Suche weiterhin auf der Projektliste ist.
118 [SELENIUM] Suchfeld wird nach der Suche geprüft: Der eingegebene Projektname muss
    weiterhin enthalten sein.
119 [SELENIUM] Suchergebnis wird geprüft: Das gesuchte Projekt muss in der Ergebnistabelle
    vorhanden sein.
120 [SELENIUM] Prüfung der Projektsuchergebnisse gestartet: Erwartetes Projekt: Selenium
    Testprojekt
121 [SELENIUM] Ergebnistabelle '#searchResult' wird gesucht und auf Sichtbarkeit geprüft.
122 [SELENIUM] Tabellenkörper der Ergebnistabelle wird erwartet.
123 [SELENIUM] Passende Tabellenzeile mit dem exakten Projektnamen wird gesucht: Selenium
    Testprojekt
124 [SELENIUM] Gefundener Projektname wird gegen den erwarteten Projektnamen geprüft.
125 [SELENIUM] Projekt wurde in der Ergebnistabelle gefunden und erfolgreich geprüft:
    Selenium Testprojekt
126 [SELENIUM] Projektsuche erfolgreich abgeschlossen. Projektname: Selenium Testprojekt
127 [SELENIUM] Projekt wird aus dem Suchergebnis geöffnet und auf korrekte Anzeige geprüft:
    Selenium Testprojekt
128 [SELENIUM] Projekt wird aus dem Suchergebnis geöffnet: Ergebniszeile und Öffnen-Link
    werden gesucht. Projektname: Selenium Testprojekt
129 [SELENIUM] Ergebniszeile für das gesuchte Projekt wird gesucht.
130 [SELENIUM] Öffnen-Link in der Ergebniszeile wird gesucht.
131 [SELENIUM] Öffnen-Link wird in den sichtbaren Bereich gescrollt.
132 [SELENIUM] Projekt wird über den Öffnen-Link geöffnet. Falls der normale Klick fehlschlä
    gt, wird JavaScript verwendet.
133 [SELENIUM] Warten, bis die Projektansicht geöffnet ist und die Suchseite verlassen wurde
    .
134 [SELENIUM] Sichtbarkeit eines Elements wird geprüft. Locator: By.id: searchResult
135 [SELENIUM] Sichtbarkeitsprüfung abgeschlossen. Locator: By.id: searchResult, sichtbar:
    false
136 [SELENIUM] Sichtbarkeit eines Elements wird geprüft. Locator: By.id: query_name
137 [SELENIUM] Sichtbarkeitsprüfung abgeschlossen. Locator: By.id: query_name, sichtbar:
    false
138 [SELENIUM] Projekt wurde erfolgreich aus dem Suchergebnis geöffnet: Selenium Testprojekt
139 [SELENIUM] Link zur Activity-Übersicht wird gesucht.
140 [SELENIUM] Activity-Übersicht wird geöffnet.
141 [SELENIUM] Warten auf vorhandene Activity-Einträge in der Übersicht.
142 [SELENIUM] Erste vorhandene Activity wird aus der Übersicht geöffnet.
143 [SELENIUM] Activity wurde erfolgreich aus dem Projekt geöffnet.
144 [SELENIUM] Bearbeitungsmodus wird gestartet: Bearbeiten-Button wird gesucht, sichtbar
    positioniert und geklickt.
145 [SELENIUM] Bearbeiten-Button wird in den sichtbaren Bereich gescrollt.
146 [SELENIUM] Bearbeiten-Button wird per JavaScript geklickt.
147 [SELENIUM] Bearbeitungsmodus wurde erfolgreich geöffnet.
148 [SELENIUM] Fortschritt wird auf Standardwert 0 zurückgesetzt.
149 [SELENIUM] Zahlenfeld wird befüllt. Locator: By.id: activity.progress, Wert: 0
150 [SELENIUM] Feld wird befüllt: Element wird gesucht. Locator: By.id: activity.progress,
    Wert: 0

```

```

151 [SELENIUM] Feld wird geleert. Locator: By.id: activity.progress
152 [SELENIUM] Wert wird in das Feld geschrieben. Locator: By.id: activity.progress, Wert: 0
153 [SELENIUM] Feld wurde erfolgreich befüllt. Locator: By.id: activity.progress
154 [SELENIUM] Zahlenfeld wurde erfolgreich befüllt. Locator: By.id: activity.progress, Wert
    : 0
155 [SELENIUM] Offene Kosten werden auf Standardwert 0 zurückgesetzt.
156 [SELENIUM] Zahlenfeld wird befüllt. Locator: By.id: activityRemainingCosts, Wert: 0
157 [SELENIUM] Feld wird befüllt: Element wird gesucht. Locator: By.id:
    activityRemainingCosts, Wert: 0
158 [SELENIUM] Feld wird geleert. Locator: By.id: activityRemainingCosts
159 [SELENIUM] Wert wird in das Feld geschrieben. Locator: By.id: activityRemainingCosts,
    Wert: 0
160 [SELENIUM] Feld wurde erfolgreich befüllt. Locator: By.id: activityRemainingCosts
161 [SELENIUM] Zahlenfeld wurde erfolgreich befüllt. Locator: By.id: activityRemainingCosts,
    Wert: 0
162 [SELENIUM] Tatsächliches Enddatum wird auf den definierten Testwert zurückgesetzt:
    17.08.2026.
163 [SELENIUM] Feld wird befüllt: Element wird gesucht. Locator: By.id: activity.
    actualEndDate, Wert: 17.08.2026
164 [SELENIUM] Feld wird geleert. Locator: By.id: activity.actualEndDate
165 [SELENIUM] Wert wird in das Feld geschrieben. Locator: By.id: activity.actualEndDate,
    Wert: 17.08.2026
166 [SELENIUM] Feld wurde erfolgreich befüllt. Locator: By.id: activity.actualEndDate
167 [SELENIUM] Zurückgesetzte Activity wird gespeichert.
168 [SELENIUM] Activity speichern gestartet: Speichern-Button wird gesucht, sichtbar
    positioniert und geklickt.
169 [SELENIUM] Speichern-Button wird in den sichtbaren Bereich gescrollt.
170 [SELENIUM] Speichern-Button wird per JavaScript geklickt.
171 [SELENIUM] Activity wurde gespeichert.
172 [SELENIUM] Reset der Activity-Testdaten erfolgreich abgeschlossen.
173 [SELENIUM] Cleanup abgeschlossen: Activity-Testdaten wurden zurückgesetzt.

```

Quellcode 9: Selenium – Kontrolllauf Activity bearbeiten (Erfolgreich)

C.2 Testprotokolle: Cypress

```
1  visit/login
2  getInput[name="username"]
3  typeMarcel Licht
4  getInput[name="password"]
5  getform#mainForm button[type="submit"]
6  click
7  url
8  assertexpected http://localhost:8082/puma/home to not include /login
9  (new url)http://localhost:8082/puma/home
10 (xhr)GET 200 /puma/widgets/projectajax?action=loadFavouritesMenu
11 (xhr)GET 200 /puma/widgets/activityajax?formatterType=HOME&projectsOnly=true&timeFilter=
    CURRENT&userIdForFavouriteActivities=667f55b2b71aed06733fe8c3&_=1786525710319
12 (xhr)GET 200 /puma/widgets/projectajax?action=loadProjectsHome&formatterType=
    CURRENT_PROJECTS&statusFilter=&_=1786525710320
13 9 get#mainForm
14 10 assertexpected #mainForm not to exist in the DOM
```

Quellcode 10: Cypress – Login (Erfolgreich)

```
1  session["Marcel Licht"]
2  created
3  14 visit/home
4  15 get#project-dropdown-toggle
5  16 click
6  (xhr)GET 200 /puma/widgets/projectajax?action=loadFavouritesMenu
7  (xhr)GET 200 /puma/widgets/activityajax?formatterType=HOME&projectsOnly=true&timeFilter=
    CURRENT&userIdForFavouriteActivities=667f55b2b71aed06733fe8c3&_=1786525976825
8  17 get#project-dropdown-menu a[href="createProject"]
9  18 click
10 19 url
11 20 assertexpected http://localhost:8082/puma/createProject to include createProject
12 (xhr)GET /puma/widgets/projectajax?action=loadProjectsHome&formatterType=
    CURRENT_PROJECTS&statusFilter=&_=1786525976826
13 21 get#project_name
14 (new url)http://localhost:8082/puma/createProject
15 22 typeCypress Testprojekt
16 (xhr)GET 200 /puma/widgets/projecttypeajax?locale=de_DE&divisionIds=
17 (xhr)GET 200 /puma/widgets/projectcategoryajax?locale=de_DE&divisionIds=&projectType=
18 (xhr)GET 200 /puma/widgets/projectssubcategoryajax?locale=de_DE&projectCategory=
19 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=PROFIT_CENTER&term=&withKeyPrefix=
    true&sortByKey=true
20 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=SHARE_CATEGORY
21 (xhr)GET 200 /puma/widgets/userprojectroleajax?action=LOAD_PREDEFINED_RECEIVERS&project.
    id=
22 (xhr)GET 200 /puma/widgets/projectcategoryajax?locale=de_DE&divisionIds=&projectType=
23 (xhr)GET 200 /puma/widgets/projectssubcategoryajax?locale=de_DE&divisionIds=&
    projectCategory=
24 23 get#project_divisionId
25 24 scrollIntoView
26 25 select{force: true}
27 26 get#project_sapProjectProfile
28 27 select{force: true}
29 (xhr)GET 200 /puma/widgets/projecttypeajax?locale=de_DE&divisionIds=576
    a3663a77320dc361b41e1
30 (xhr)GET 200 /puma/widgets/fieldvaluesajax?project.id=&divisionId=576
    a3663a77320dc361b41e1
31 28 get#project_sapProfitCenter
32 29 select{force: true}
33 30 get#project_endDate
34 31 scrollIntoView
35 32 clear
36 33 type17.08.2026
37 34 get#project_sponsor_id
38 35 next.select2
39 36 find.select2-selection--single
40 37 scrollIntoView
41 38 click{force: true}
42 (xhr)GET 200 /puma/widgets/durationajax?endDate=2026-08-17T00%3A00%3A00.000Z&startDate
    =2026-08-12T00%3A00%3A00.000Z
43 39 get.select2-search__field:visible
44 40 assertexpected <input.select2-search__field> to have a length of 1
```

```

45 41 typeMarcel Licht
46 (xhr)GET 200 /puma/widgets/projectcategoryajax?locale=de_DE&divisionIds=576
    a3663a77320dc361b41e1&projectType=
47 (xhr)GET 200 /puma/widgets/projectssubcategoryajax?locale=de_DE&divisionIds=576
    a3663a77320dc361b41e1&projectCategory=
48 42 contains.select2-results__option, Marcel Licht
49 43 assertexpected <li.select2-results__option.select2-results__option--highlighted> to
    be visible
50 (xhr)GET 200 /puma/widgets/userajax?term=Marcel%20Licht
51 44 click
52 45 get#project_manager_id
53 46 next.select2
54 47 find.select2-selection--single
55 48 scrollIntoView
56 49 click{force: true}
57 50 get.select2-search__field:visible
58 51 assertexpected <input.select2-search__field> to have a length of 1
59 52 typeMarcel Licht
60 53 contains.select2-results__option, Marcel Licht
61 54 assertexpected <li.select2-results__option.select2-results__option--highlighted> to
    be visible
62 (xhr)GET 200 /puma/widgets/userajax?term=Marcel%20Licht
63 55 click
64 56 get#project_projectType
65 57 select{force: true}
66 58 wait@projectCategoryAjax1
67 (xhr)GET 200 /puma/widgets/projectcategoryajax?locale=de_DE&divisionIds=576
    a3663a77320dc361b41e1&projectType=IMPROVEMENT
68 projectCategoryAjax
69 (xhr)GET 200 /puma/widgets/projectssubcategoryajax?locale=de_DE&divisionIds=576
    a3663a77320dc361b41e1&projectCategory=
70 projectSubcategoryAjax
71 59 get#project_projectCategory
72 60 select{force: true}
73 61 getiframe.cke_wysiwyg_frame
74 62 filter:visible
75 63 first
76 64 its.0.contentDocument.body
77 65 assertexpected <body.cke_editable.cke_editable_themed.cke_contents_ltr.
    cke_show_borders> not to be empty
78 66 wrap<body.cke_editable.cke_editable_themed.cke_contents_ltr.cke_show_borders>
79 67 findp
80 68 first
81 69 click
82 (xhr)GET 200 /puma/widgets/projectssubcategoryajax?locale=de_DE&divisionIds=576
    a3663a77320dc361b41e1&projectCategory=OPTIMIZATION
83 projectSubcategoryAjax
84 70 typeDieses Testprojekt dient im Rahmen einer Fallstudie zur Erprobung und Bewertung
    von automatisierten Tests mit Cypress. Ziel ist es, typische Eingabe- und Prü
    fprozesse in der Anwendung realitätsnah nachzustellen und die Stabilität, Zuverlä
    ssigkeit sowie Benutzerfreundlichkeit der Testautomatisierung zu untersuchen. Die
    Ergebnisse sollen als Grundlage für eine Nutzwertanalyse dienen, um den praktischen
    Mehrwert von Cypress im Vergleich zu Selenium zu bewerten., {force: true}
85 71 containsbutton, Speichern
86 72 click{force: true}
87 (xhr)POST 200 /puma/widgets/projectajax?action=loadRelatedProjects&formatterType=
    RELATED_PROJECTS
88 (new url)http://localhost:8082/puma/projectEditor?project.id=6a7c393072e2b75d4fc9f518&
    projectTypes=PRODUCT&projectTypes=PRODUCT_MAINTENANCE&projectTypes=INVESTMENT&
    projectTypes=COMPLAINT&projectTypes=IMPROVEMENT&projectTypes=FIXED_COSTS&
    projectTypes=TECHNOLOGY_PROJECT&projectCategories=DEVELOPMENT&projectCategories=
    MINOR&projectCategories=MAJOR&projectCategories=SUSPENSION&projectCategories=
    COLLECTING&projectCategories=RIGHTS&projectCategories=LICENSES&projectCategories=
    COMMON&projectCategories=DRAWING_CHANGE&projectCategories=PURCHASING&
    projectCategories=RENOVATION&projectCategories=RESOURCES&projectCategories=
    PROPERTIES&projectCategories=INTERNAL&projectCategories=EXTERNAL&projectCategories=
    OPTIMIZATION&projectCategories=ORGANISATION&projectCategories=MARKETING&
    projectSubCategories=JUMO_PRODUCT&projectSubCategories=CUSTOMER_PRODUCT&
    projectSubCategories=TECHNOLOGY_IDEA&projectSubCategories=PRODCUT_IDEA&
    projectSubCategories=COMPETITION_ANALYSIS&projectSubCategories=RIGHTS_DESIGN&
    projectSubCategories=RIGHTS_TRADE_MARK&projectSubCategories=RIGHTS_PATENT&
    projectSubCategories=LICENSES_CERTIFICATE&projectSubCategories=NPZ&
    projectSubCategories=NEW_BEGINNINGS&projectSubCategories=SUPPLIER_QUALIFICATION&

```

```

      projectSubCategories=MODULE_DISCONTINUATION&projectSubCategories=SECOND_SOURCE
89 (xhr)GET 200 /puma/widgets/commentajax?action=LOAD_LIST&project.id=6
      a7c393072e2b75d4fc9f518&originClassName=Project&originId=6a7c393072e2b75d4fc9f518
90 (xhr)GET 200 /puma/widgets/projectcategoryajax?locale=de_DE&divisionIds=576
      a3663a77320dc361b41e1&projectType=IMPROVEMENT
91 projectCategoryAjax
92 (xhr)GET 200 /puma/widgets/projecttypeajax?locale=de_DE&divisionIds=576
      a3663a77320dc361b41e1
93 (xhr)GET 200 /puma/widgets/projectssubcategoryajax?locale=de_DE&projectCategory=
      OPTIMIZATION
94 projectSubcategoryAjax
95 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=PROFIT_CENTER&term=&withKeyPrefix=
      true&sortByKey=true
96 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=SHARE_CATEGORY&divisionId=576
      a3663a77320dc361b41e1
97 (xhr)GET 200 /puma/widgets/userprojectroleajax?action=LOAD_PREDEFINED_RECEIVERS&project.
      id=6a7c393072e2b75d4fc9f518
98 (xhr)GET 200 /puma/widgets/foldersajax?project.id=6a7c393072e2b75d4fc9f518
99 (xhr)GET 200 /puma/widgets/projectscategoryajax?locale=de_DE&divisionIds=576
      a3663a77320dc361b41e1&projectType=IMPROVEMENT
100 projectCategoryAjax
101 (xhr)GET 200 /puma/widgets/documentsajax?formatType=EXTERNAL&projectId=6
      a7c393072e2b75d4fc9f518&showInProjectEditor=true&_=1786526000914
102 (xhr)GET 200 /puma/widgets/projectssubcategoryajax?locale=de_DE&divisionIds=576
      a3663a77320dc361b41e1&projectCategory=OPTIMIZATION

```

Quellcode 11: Cypress – Neues Projekt (Erfolgreich)

```

1 1 session["Marcel Licht"]
2 created
3 14 visit/home
4 15 get#project-dropdown-toggle
5 16 click
6 (xhr)GET 200 /puma/widgets/projectajax?action=loadFavouritesMenu
7 17 get#project-dropdown-menu a[href="projectList"]
8 18 click
9 19 url
10 20 assertexpected http://localhost:8082/puma/projectList to include projectList
11 (xhr)GET /puma/widgets/projectajax?action=loadProjectsHome&formatterType=
      CURRENT_PROJECTS&statusFilter=&_=1786526049987
12 (xhr)GET /puma/widgets/activityajax?formatterType=HOME&projectsOnly=true&timeFilter=
      CURRENT&userIdForFavouriteActivities=667f55b2b71aed06733fe8c3&_=1786526049986
13 (new url)http://localhost:8082/puma/projectList
14 21 get#query_name
15 22 clear
16 23 typeCypress Testprojekt
17 24 containsbutton, Suchen
18 25 click{force: true}
19 (xhr)POST 200 /puma/widgets/projectajax?action=loadProjects&formatterType=SEARCH_RESULT
20 26 contains#searchResult tbody tr, Cypress Testprojekt
21 27 finda.openEntry
22 28 click
23 29 get#createNewActivity
24 (new url)http://localhost:8082/puma/projectOverview?project.id=6a7c393072e2b75d4fc9f518
25 30 click{force: true}
26 31 getinput[id="activity.title"]
27 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=SHARE_CATEGORY&divisionId=576
      a3663a77320dc361b41e1
28 (xhr)GET 200 /puma/widgets/userprojectroleajax?action=LOAD_PREDEFINED_RECEIVERS&project.
      id=6a7c393072e2b75d4fc9f518
29 (xhr)GET /puma/widgets/memberajax?formatterType=MEMBERLIST_OVERVIEW&action=loadMembers&
      project.id=6a7c393072e2b75d4fc9f518&_=1786526054036
30 (xhr)GET /puma/widgets/activityajax?formatterType=ACTIVITIES_OVERVIEW&projectId=6
      a7c393072e2b75d4fc9f518&type=&_=1786526054037
31 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=ACTIVITY_WORKPLACE&term=&
      withKeyPrefix=true
32 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=COSTPLANNING_MATERIALGROUP&term=&
      withKeyPrefix=true
33 (xhr)GET 200 /puma/widgets/lock?objectId=6a7c393072e2b75d4fc9f518&type=PROJECT
34 (xhr)POST 200 /puma/widgets/lock
35 (new url)http://localhost:8082/puma/activityEditor?action=createNew&project.id=6
      a7c393072e2b75d4fc9f518
36 32 scrollIntoView

```



```

37 33 clear
38 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=SHARE_CATEGORY&divisionId=576
    a3663a77320dc361b41e1
39 (xhr)GET 200 /puma/widgets/userprojectroleajax?action=LOAD_PREDEFINED_RECEIVERS&project.
    id=6a7c393072e2b75d4fc9f518
40 34 typeTest Projekt Plan
41 35 getiframe.cke_wysiwyg_frame
42 36 filter:visible
43 37 first
44 38 its.0.contentDocument.body
45 39 assertexpected <body.cke_editable.cke_editable_themed.cke_contents_ltr.
    cke_show_borders> not to be empty
46 40 wrap<body.cke_editable.cke_editable_themed.cke_contents_ltr.cke_show_borders>
47 41 findp
48 42 first
49 43 click
50 44 typeDieses Testprojekt dient im Rahmen einer Fallstudie zur Erprobung und Bewertung
    automatisierter Tests mit Cypress. Ziel ist es, typische Eingabe- und Prüfprozesse
    in der Anwendung realitätsnah nachzustellen sowie die Stabilität, Zuverlässigkeit
    und Benutzerfreundlichkeit der Testautomatisierung zu untersuchen. Die Ergebnisse
    sollen als Grundlage für eine Nutzwertanalyse dienen, um den praktischen Mehrwert
    von Cypress im Vergleich zu Selenium zu bewerten., {force: true}
51 45 scrollIntoView
52 46 getinput[id="activityStart"]
53 47 scrollIntoView
54 48 clear
55 49 type13.08.2026
56 50 getinput[id="activityEnd"]
57 51 scrollIntoView
58 52 clear
59 53 type17.08.2026
60 (xhr)GET 200 /puma/widgets/durationajax?startDate=2026-08-12T22%3A00%3A00.000Z&endDate
    =2026-08-16T22%3A00%3A00.000Z
61 54 getinput[id="activityEstimation"]
62 55 scrollIntoView
63 56 clear
64 57 type14
65 58 get#activity_workplace
66 59 select{force: true}
67 60 getinput[id="activity.actualStartDate"]
68 61 scrollIntoView
69 62 clear
70 63 type15.06.2026
71 64 getinput[id="activity.actualEndDate"]
72 65 scrollIntoView
73 66 clear
74 67 type17.08.2026
75 68 getinput[id="activityRemainingHours"]
76 69 scrollIntoView
77 70 clear
78 71 type13
79 72 getinput[id="activity.progress"]
80 73 scrollIntoView
81 74 clear
82 75 type4
83 76 getinput[id="activityRemainingCosts"]
84 77 scrollIntoView
85 78 clear
86 79 type0
87 80 containsbutton, Speichern
88 81 click{force: true}
89 (xhr)POST 200 /puma/widgets/lock
90 (xhr)GET 200 /puma/widgets/commentajax?action=LOAD_LIST&project.id=6
    a7c393072e2b75d4fc9f518&originClassName=Activity&originId=6a7c397672e2b75d4fc9f51c
91 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=ACTIVITY_WORKPLACE&term=&
    withKeyPrefix=true
92 (new url)http://localhost:8082/puma/activityEditor?project.id=6a7c393072e2b75d4fc9f518&
    id=6a7c397672e2b75d4fc9f51c
93 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=COSTPLANNING_MATERIALGROUP&term=&
    withKeyPrefix=true
94 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=SHARE_CATEGORY&divisionId=576
    a3663a77320dc361b41e1
95 (xhr)GET 200 /puma/widgets/userprojectroleajax?action=LOAD_PREDEFINED_RECEIVERS&project.

```

```

id=6a7c393072e2b75d4fc9f518
96 (xhr)GET 200 /puma/widgets/foldersajax?project.id=6a7c393072e2b75d4fc9f518
97 (xhr)GET 200 /puma/widgets/documentsajax?formatType=EXTERNAL&activityIds=6
a7c397672e2b75d4fc9f51c&_=1786526070932

```

Quellcode 12: Cypress – Neue Activity (Erfolgreich)

```

1  session["Marcel Licht"]
2  created
3  14  visit/home
4  15  get#project-dropdown-toggle
5  16  click
6  (xhr)GET 200 /puma/widgets/projectajax?action=loadFavouritesMenu
7  17  get#project-dropdown-menu a[href="projectList"]
8  18  click
9  19  url
10 20  assertexpected http://localhost:8082/puma/projectList to include projectList
11 (xhr)GET 200 /puma/widgets/activityajax?formatterType=HOME&projectsOnly=true&timeFilter=
CURRENT&userIdForFavouriteActivities=667f55b2b71aed06733fe8c3&_=1786526170442
12 (xhr)GET /puma/widgets/projectajax?action=loadProjectsHome&formatterType=
CURRENT_PROJECTS&statusFilter=&_=1786526170443
13 (new url)http://localhost:8082/puma/projectList
14 21  get#query_name
15 22  clear
16 23  typeCypress Testprojekt
17 24  containsbutton, Suchen
18 25  click{force: true}
19 (xhr)POST 200 /puma/widgets/projectajax?action=loadProjects&formatterType=SEARCH_RESULT
20 26  contains#searchResult tbody tr, Cypress Testprojekt
21 27  finda.openEntry
22 28  click
23 29  containsa[href*="activityList"], Übersicht
24 (new url)http://localhost:8082/puma/projectOverview?project.id=6a7c393072e2b75d4fc9f518
25 30  click{force: true}
26 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=SHARE_CATEGORY&divisionId=576
a3663a77320dc361b41e1
27 (xhr)GET 200 /puma/widgets/userprojectroleajax?action=LOAD_PREDEFINED_RECEIVERS&project.
id=6a7c393072e2b75d4fc9f518
28 31  geta.openEntry[title="Eintrag öffnen"]
29 (xhr)GET 200 /puma/widgets/memberajax?formatterType=MEMBERLIST_OVERVIEW&action=
loadMembers&project.id=6a7c393072e2b75d4fc9f518&_=1786526174230
30 (xhr)GET /puma/widgets/activityajax?formatterType=ACTIVITIES_OVERVIEW&projectId=6
a7c393072e2b75d4fc9f518&type=&_=1786526174231
31 (xhr)GET 200 /puma/widgets/activityajax?action=ACTIVITYCOPYPOSITIONS
32 (new url)http://localhost:8082/puma/activityList?project.id=6a7c393072e2b75d4fc9f518
33 (xhr)GET 200 /puma/widgets/activityajax?formatterType=DEFAULT&projectId=6
a7c393072e2b75d4fc9f518&type=&_=1786526175381
34 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=SHARE_CATEGORY&divisionId=576
a3663a77320dc361b41e1
35 (xhr)GET 200 /puma/widgets/userprojectroleajax?action=LOAD_PREDEFINED_RECEIVERS&project.
id=6a7c393072e2b75d4fc9f518
36 32  click
37 33  get#activity.progress
38 34  assertexpected <input#activity.progress.form-control> to have value '15'
39 (xhr)GET 200 /puma/widgets/commentajax?action=LOAD_LIST&project.id=6
a7c393072e2b75d4fc9f518&originClassName=Activity&originId=6a7c397672e2b75d4fc9f51c
40 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=ACTIVITY_WORKPLACE&term=&
withKeyPrefix=true
41 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=COSTPLANNING_MATERIALGROUP&term=&
withKeyPrefix=true
42 (new url)http://localhost:8082/puma/activityEditor?project.id=6a7c393072e2b75d4fc9f518&
id=6a7c397672e2b75d4fc9f51c
43 35  get#activity.actualEndDate
44 36  assertexpected <input#activity.actualEndDate.form-control> to have value 17.08.2026
45 37  get#activityRemainingCosts
46 38  assertexpected <input#activityRemainingCosts.form-control> to have value '50'
47 39  get#activity_workplace
48 40  assertexpected <select#activity_workplace.form-control.prefetch-select.select2-
hidden-accessible> to have value PS02080
49 41  gettr.task .task_description
50 42  eq0
51 43  assertexpected <textarea.form-control.task_description> to have value Alle Testfälle
für Cypress implementieren

```

```

52 44 gettr.task.task_description
53 45 eq1
54 46 assertexpected <textarea.form-control.task_description> to have value Mit
    schriftlicher Ausarbeitung der Fallstudie beginnen
55 47 containsbutton, Bearbeiten
56 48 click{force: true}
57 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=SHARE_CATEGORY&divisionId=576
    a3663a77320dc361b41e1
58 (xhr)GET 200 /puma/widgets/lock?objectId=6a7c393072e2b75d4fc9f518&type=PROJECT
59 (xhr)GET 200 /puma/widgets/userprojectroleajax?action=LOAD_PREDEFINED_RECEIVERS&project.
    id=6a7c393072e2b75d4fc9f518
60 (xhr)GET 200 /puma/widgets/foldersajax?project.id=6a7c393072e2b75d4fc9f518
61 (xhr)GET 200 /puma/widgets/documentsajax?formatType=EXTERNAL&activityIds=6
    a7c397672e2b75d4fc9f51c&_id=1786526177300
62 (xhr)POST 200 /puma/widgets/lock
63 49 get#activity.progress
64 50 scrollIntoView
65 51 clear
66 52 type4
67 53 get#activity.actualEndDate
68 54 scrollIntoView
69 55 clear
70 56 type17.08.2026
71 57 get#activity.RemainingCosts
72 58 scrollIntoView
73 59 type0, {force: true}
74 60 get#activity_workplace
75 61 select{force: true}
76 62 getbody
77 63 wrap[ <tr.task.odd>, 1 more... ]
78 64 first
79 65 find.deleteEntry
80 66 click{force: true}
81 67 containsbutton, Löschen
82 68 assertexpected <button.btn.btn-danger.btn-ok> to be visible
83 69 click{force: true}
84 70 getbody
85 71 wrap<tr.task.odd>
86 72 first
87 73 find.deleteEntry
88 74 click{force: true}
89 75 containsbutton, Löschen
90 76 assertexpected <button.btn.btn-danger.btn-ok> to be visible
91 77 click{force: true}
92 78 getbody
93 79 logKeine Tasks mehr vorhanden
94 80 containsbutton, Speichern
95 81 click{force: true}
96 (xhr)POST 200 /puma/widgets/lock
97 (xhr)GET 200 /puma/widgets/commentajax?action=LOAD_LIST&project.id=6
    a7c393072e2b75d4fc9f518&originClassName=Activity&originId=6a7c397672e2b75d4fc9f51c
98 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=ACTIVITY_WORKPLACE&term=&
    withKeyPrefix=true
99 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=COSTPLANNING_MATERIALGROUP&term=&
    withKeyPrefix=true
100 (xhr)GET 200 /puma/widgets/keyajax?keyContext.context=SHARE_CATEGORY&divisionId=576
    a3663a77320dc361b41e1
101 (xhr)GET 200 /puma/widgets/userprojectroleajax?action=LOAD_PREDEFINED_RECEIVERS&project.
    id=6a7c393072e2b75d4fc9f518
102 (xhr)GET 200 /puma/widgets/foldersajax?project.id=6a7c393072e2b75d4fc9f518
103 (xhr)GET 200 /puma/widgets/documentsajax?formatType=EXTERNAL&activityIds=6
    a7c397672e2b75d4fc9f51c&_id=1786526183348
104 (xhr)GET 200 /puma/widgets/projectajax?action=loadFavouritesMenu

```

Quellcode 13: Cypress – Activity bearbeiten & kontrollieren (Erfolgreich)